SISTEMA COLABORATIVO DE ROBOTS MÓVILES PARA ASISTENCIA DE ORIENTACIÓN EN ENTORNOS INTERIORES CON MÚLTIPLES PISOS

Santiago Hernández Ávila

Jonny Alejandro Mejía León

Modalidad de Proyecto de Grado
(Proyecto de Investigación)

GED — Grupo de Estudio y Desarrollo en Robótica
(Robótica y Sistemas Autónomos)



UNIVERSIDAD SANTO TOMÁS
PRIMER CLAUSTRO UNIVERSITARIO DE COLOMBIA

UNIVERSIDAD SANTO TOMÁS
FACULTAD DE INGENIERÍA ELECTRÓNICA
PROGRAMA DE INGENIERÍA ELECTRÓNICA
Bogotá D.C.
2026

SISTEMA COLABORATIVO DE ROBOTS MÓVILES PARA ASISTENCIA DE ORIENTACIÓN EN ENTORNOS INTERIORES CON MÚLTIPLES PISOS

Santiago Hernández Ávila

Jonny Alejandro Mejía León

Proyecto de grado presentado como requisito para optar al título de:
Ingeniero Electrónico

DIRECTOR DEL PROYECTO
Ing. Armando Mateus Rojas, MSc.

UNIVERSIDAD SANTO TOMÁS
FACULTAD DE INGENIERÍA ELECTRÓNICA
PROGRAMA DE INGENIERÍA ELECTRÓNICA
Bogotá D.C.
2026

AUTORIDADES ACADÉMICAS

P. NOMBRES COMPLETOS, O.P.
Rector General

P. NOMBRES COMPLETOS, O.P.
Vicerrector Administrativo y Financiero General

P. NOMBRES COMPLETOS, O.P.
Vicerrector Académico General

NOMBRE COMPLETO
Secretaria General

P. NOMBRES COMPLETOS, O.P.
Director de División de Ingenierías

NOMBRE COMPLETO
Secretaria de División de Ingenierías

NOMBRE COMPLETO
Decano Facultad de Ingeniería Electrónica

Nota de aceptación:

_____

_____

_____

_____

_____

Firma del Director

_____

Firma del Jurado 1

_____

Firma del Jurado 2

Bogotá D.C., Colombia
octubre de 2026

ADVERTENCIA

La Universidad Santo Tomás no se hace responsable de las opiniones y conceptos expresados en el trabajo de grado, solo velará por qué no se publique nada contrario al dogma ni a la moral católica y porque el trabajo no tenga ataques personales y únicamente se vea el anhelo de buscar la verdad científica.

AGRADECIMIENTOS

Al ingeniero Armando Mateus Rojas, director del proyecto, por el seguimiento semanal y por insistir en que cada afirmación del documento tuviera detrás un dato que la sostuviera.

Al Grupo de Estudio y Desarrollo en Robótica de la Universidad Santo Tomás, por el espacio de trabajo y por los dos vehículos AWS DeepRacer. Sin ellos el proyecto habría tenido que empezar por construir el hardware.

A los docentes y compañeros de la Facultad de Ingeniería Electrónica que prestaron equipo, acompañaron las jornadas de medición en los pasillos y leyeron borradores de este documento.

A nuestras familias, por el respaldo durante los dos semestres que tomó el trabajo.

Conflicto de intereses

Los autores declaran no tener ningún conflicto de intereses. Los patrocinadores no participaron en el diseño del estudio, la recopilación, el análisis ni la interpretación de los datos publicados, ni en la redacción del documento.

RESUMEN

Orientarse en un edificio de varios pisos es difícil para quien no lo conoce, y el cambio de nivel corta la continuidad del recorrido. Las soluciones robóticas existentes resuelven ese salto de dos maneras: dotando al robot de mecanismos para subir escaleras, o conectándolo al ascensor. Las dos son caras y obligan a intervenir el edificio.

Este trabajo propone y evalúa una alternativa. Dos robots móviles terrestres, uno por planta, se reparten el guiado de una persona mediante un protocolo de relevo en la zona de escaleras. Ningún vehículo cruza de nivel. Un coordinador central asigna el agente según el nivel del origen y del destino. El usuario pide el servicio desde una interfaz web que funciona en cualquier teléfono, sin instalar nada y sin salida a internet.

La evaluación siguió un protocolo escrito antes de instrumentar el sistema, con cuatro métricas de definición operativa. En simulación, treinta misiones sorteadas dieron una campaña válida sin descartes, que permite afirmar con 95 % de confianza que la tasa de éxito supera el 70 %.

De diez misiones sorteadas sobre dos vehículos reales, uno por piso, nueve se completaron con relevo, con un tiempo total mediano de 87,6 s y la llegada a menos de 0,5 m del destino según la localización de cada vehículo. La décima se interrumpió porque el computador del coordinador se quedó sin batería. La precisión de llegada está limitada por la plataforma, que no se mueve por debajo de 0,40 m/s.

Palabras clave: robótica móvil colaborativa; sistemas multi-robot; navegación en interiores; protocolo de relevo; interacción humano-robot; ROS 2.

ABSTRACT

Finding one's way inside a multi-storey building is hard for first-time visitors, and changing floors breaks the continuity of the route. Existing robotic solutions address that gap in two ways: by equipping the robot to climb stairs, or by interfacing it with the building lift. Both are expensive and require modifying the infrastructure.

This work proposes and evaluates an alternative. Two ground mobile robots, one per floor, share the task of guiding a person through a handover protocol at the stairwell. Neither vehicle crosses between levels. A central coordinator assigns the agent according to the floor of the origin and the destination. The user requests the service from a web interface that runs on any phone, with no installation and no internet access.

The evaluation followed a protocol written before instrumenting the system, with four operationally defined metrics. In simulation, thirty randomised missions gave a valid campaign with no discards, which supports, with 95 % confidence, the claim that the success rate exceeds 70 %.

Of ten randomised missions on two real vehicles, one per floor, nine were completed with a handover, with a median total time of 87.6 s and arrival within 0.5 m of the goal according to each vehicle's own localisation. The tenth was interrupted when the coordinator's computer ran out of battery. Arrival accuracy is limited by the platform, which does not move below 0.40 m/s.

Keywords: collaborative mobile robotics; multi-robot systems; indoor navigation; handover protocol; human-robot interaction; ROS 2.

GLOSARIO

Los términos siguientes se usan desde el resumen y la introducción. El capítulo 2 los desarrolla y añade los demás.

Agente
: Cada uno de los dos robots del sistema. Se usa «agente» y no «robot» cuando importa su papel dentro de la coordinación y no su hardware.

AMCL
: Algoritmo de localización por filtro de partículas. Combina la odometría con las lecturas del láser para situar al robot dentro de un mapa que ya conoce. No construye el mapa: lo supone dado.

Banda muerta de tracción
: Rango de velocidades ordenadas en el que el vehículo no llega a moverse, porque la señal resultante no vence el rozamiento estático. En la plataforma de este trabajo abarca todo lo que está por debajo de 0,40 m/s, y condiciona la aproximación final del vehículo al destino.

Cinemática tipo automóvil (dirección Ackermann)
: Modelo de un vehículo que dirige con el eje delantero. Tiene radio de giro mínimo y no puede girar sobre su propio eje. Es lo contrario de la tracción diferencial, que es lo que la mayoría de las herramientas de navegación supone por omisión.

Continuidad del servicio
: Propiedad de no interrumpir la asistencia mientras el usuario cambia de nivel. Es el objeto de la pregunta problema.

Localización
: Estimación de la posición y la orientación del robot respecto a un marco de referencia fijo.

Mapa de ocupación
: Cuadrícula en la que cada celda se declara libre, ocupada o desconocida. Es la representación del edificio con la que funcionan la localización y la planificación.

Mapa de costos (*costmap*)
: Cuadrícula derivada de la anterior en la que cada celda lleva un costo de tránsito en lugar de una sola etiqueta. Permite que el planificador prefiera el centro del pasillo sin prohibir los bordes.

Middleware robótico
: Capa de software que comunica entre sí los módulos del sistema. Aquí es ROS 2.

Misión

Una solicitud de guiado completa, desde que el usuario elige origen y destino hasta que el sistema la cierra. Es la unidad de medida de la campaña experimental: las métricas se reportan por misión.

Observabilidad

Propiedad de una magnitud física que permite estimarla a partir de las mediciones disponibles. Si en cierta combinación de entorno y sensor una magnitud es inobservable, ningún ajuste de parámetros la recupera, porque la información no está en la señal. La sección 2.3.5 muestra un caso concreto en un pasillo recto.

Odometría

Estimación del desplazamiento propio del robot a partir de sus propias mediciones. La plataforma de este trabajo no tiene encoders, así que la obtiene comparando barridos sucesivos del láser.

Punto de transferencia

Lugar físico, en este trabajo la escalera, donde un agente entrega el guiado y el otro lo recibe.

Relevo

Traspaso de la responsabilidad de guiar a una persona, de un agente al otro, en un punto acordado de antemano. Es el aporte que el proyecto declara, y lo que sustituye al hardware de transición vertical.

Tasa de éxito

Proporción de misiones que terminan como el protocolo experimental declara que deben terminar. Su definición operativa está en la sección 3.6.

Tiempo de respuesta

Intervalo entre la solicitud del usuario y el instante en que el robot asignado arranca la tarea.

Zona de transición

Punto del edificio donde se cambia de nivel: escalera o ascensor. En este sistema ningún robot la atraviesa.

ABREVIATURAS

AMCL

Adaptive Monte Carlo Localization — localización adaptativa por método de Monte Carlo.

AWS

Amazon Web Services, fabricante de la plataforma DeepRacer empleada como vehículo.

CONPES

Consejo Nacional de Política Económica y Social (Colombia).

DDS

Data Distribution Service — protocolo de transporte sobre el que ROS 2 implementa sus comunicaciones.

GED

Grupo de Estudio y Desarrollo en Robótica, grupo de investigación en el que se inscribe el proyecto.

GUI

Graphical User Interface — interfaz gráfica de usuario.

HRI

Human-Robot Interaction — interacción humano-robot.

ICONTEC

Instituto Colombiano de Normas Técnicas y Certificación.

IMU

Inertial Measurement Unit — unidad de medición inercial.

JSON

JavaScript Object Notation, formato de intercambio de datos en texto.

MRS

Multi-Robot System — sistema multi-robot.

Nav2

Navigation2, marco de navegación autónoma de ROS 2.

NTC

Norma Técnica Colombiana.

PID

Control proporcional-integral-derivativo.

PWM

Pulse Width Modulation — modulación por ancho de pulso.

QoS

Quality of Service — políticas de entrega de un canal de comunicación.

ROS

Robot Operating System. La versión empleada es ROS 2, que no es una actualización de ROS 1 sino un rediseño sobre DDS.

SLAM

Simultaneous Localization and Mapping — localización y mapeo simultáneos.

CONTENIDO

Autoridades académicas I

Agradecimientos IV

Resumen V

Abstract VI

Glosario VII

Abreviaturas IX

Contenido XI

Lista de figuras XV

Lista de tablas XVII

INTRODUCCIÓN 1

1. Contextualización 3

1.1. Planteamiento del problema . . . 3

1.2. Pregunta problema . . . 3

1.3. Objetivos . . . 4

1.3.1. Objetivo general . . . 4

1.3.2. Objetivos específicos . . . 4

2. Marco referencial 5

2.1. Antecedentes . . . 5

2.1.1. Robots que guían a personas . . . 5

2.1.2. Navegación autónoma en interiores . . . 6

2.1.3. Coordinación entre varios robots . . . 7

2.1.4. Movilidad entre pisos . . . 8

2.1.5. Distribución del cómputo . . . 8

2.1.6. Aporte de este trabajo frente a los antecedentes . . . 9

2.2. Marco conceptual . . . 9

2.2.1. Navegación y percepción . . . 10

2.2.2. Control y locomoción . . . 11
2.2.3. Comunicación y sistemas distribuidos . . . 12
2.2.4. Arquitectura del sistema . . . 12
2.2.5. Interacción humano-robot . . . 13
2.2.6. Métricas y criterios de evaluación . . . 13
2.3. Marco teórico . . . 14
2.3.1. Asignación de tareas en sistemas multirrobot . . . 14
2.3.2. ROS 2 y la comunicación entre los componentes . . . 15
2.3.3. Cinemática tipo automóvil y planificación de trayectorias . . . 16
2.3.4. Seguimiento de la trayectoria . . . 17
2.3.5. Localización, odometría y observabilidad . . . 18
2.3.6. Simulación del sistema . . . 19
2.3.7. La interfaz web como cliente del sistema . . . 19
2.3.8. Evaluación experimental . . . 20
2.4. Marco legal . . . 21
2.4.1. Alcance de la operación y responsabilidad . . . 21
2.5. Marco ambiental . . . 22
2.5.1. Reutilización de hardware existente . . . 22
2.5.2. Consumo energético y autonomía . . . 22
2.5.3. Ciclo de vida de las baterías . . . 23

3. Desarrollo **24**
3.1. Etapas del trabajo . . . 24
3.2. Recorrido del trabajo . . . 25
3.2.1. La plataforma DonkeyCar y su simulador . . . 26
3.2.2. El modelo de AWS DeepRacer y su migración a ROS 2 Humble . . . 27
3.2.3. Los primeros entornos simulados y el seguidor de paredes . . . 28
3.2.4. La integración en simulación . . . 29
3.2.5. Del vehículo físico a la campaña con los dos vehículos . . . 30
3.3. Arquitectura funcional del sistema . . . 31
3.3.1. Requisitos . . . 32
3.3.2. El contrato de interfaces . . . 33
3.3.3. El planificador, escrito sin ROS . . . 34
3.3.4. La asignación de tareas . . . 35
3.3.5. Escenarios de operación en simulación . . . 35
3.3.6. De la simulación a los vehículos . . . 36

3.4. Plataforma robótica . . . 38
3.4.1. Selección de la plataforma: de DonkeyCar a DeepRacer . . . 38
3.4.2. Componentes del vehículo . . . 39
3.4.3. Nav2 sobre un vehículo que no gira sobre su eje . . . 40
3.4.4. El orden de puesta en marcha . . . 41
3.4.5. La odometría del láser como limitación principal . . . 41
3.4.6. Los mapas, derivados de medidas y no de SLAM . . . 43
3.4.7. La banda muerta de tracción y lo que obliga a configurar . . . 43
3.4.8. La unidad de medición inercial de la tarjeta . . . 44
3.4.9. Los dos vehículos en la misma red . . . 45
3.4.10. La media vuelta en varios tiempos . . . 46
3.5. Interfaz de interacción con el usuario . . . 46
3.5.1. Una página web, no una aplicación . . . 46
3.5.2. Lo que el usuario ve . . . 47
3.5.3. Confirmación de cambio de piso . . . 48
3.5.4. Cancelación . . . 50
3.5.5. Ajustes tras las pruebas con los vehículos . . . 50
3.6. Protocolo experimental e instrumentación . . . 51
3.6.1. Definiciones operativas . . . 51
3.6.2. Diseño de la campaña en simulación . . . 52
3.6.3. Instrumentación . . . 53
3.6.4. Pruebas de verificación de la demostración física . . . 53

4. Resultados **55**
4.1. Evaluación en simulación . . . 55
4.1.1. Tasa de éxito . . . 55
4.1.2. Causa común de los cuatro fallos . . . 56
4.1.3. Tiempo de respuesta y tiempo de asignación . . . 56
4.2. Evaluación con los vehículos . . . 57
4.2.1. La odometría del vehículo . . . 57
4.2.2. La navegación y la precisión de llegada . . . 58
4.3. Evaluación del sistema completo con los dos vehículos . . . 60
4.3.1. Condiciones de las pruebas . . . 60
4.3.2. La misión con relevo . . . 60
4.3.3. La campaña con las misiones sorteadas . . . 61
4.3.4. El relevo entre los dos vehículos . . . 62

4.3.5. Criterio de conteo e incidencias 63
4.4. Limitaciones de la medición 63
4.4.1. Naturaleza del criterio de continuidad entre niveles 64
4.4.2. Ancho del intervalo de confianza 64
4.4.3. Espacio desconocido marcado como libre en el mapa 64
4.4.4. La llegada en los vehículos, sin medida externa 65

CONCLUSIONES **66**

TRABAJOS FUTUROS **68**

Referencias bibliográficas **70**

LISTA DE FIGURAS

3.1. Exploración de la plataforma DonkeyCar. A la izquierda, su simulador conducido desde la interfaz web de la plataforma, con la imagen de la cámara del vehículo. A la derecha, el modelo propio del vehículo en el visualizador de ROS 2, con el chasis, las ruedas y la dirección. 26
3.2. El modelo del DeepRacer migrado a ROS 2 Humble. A la izquierda, en un mundo vacío, con el barrido del láser en azul. A la derecha, dentro del primer modelo del pasillo de la universidad, de 7 por 3 m; el sector sin barrido detrás del vehículo es el que no cubría el láser del modelo simulado. 27
3.3. Arquitectura del sistema en simulación, con un proceso de Gazebo por piso. Ningún robot atraviesa la zona de transición: lo que la cruza es el usuario, y lo que se traspasa entre agentes es la responsabilidad de guiarlo. 32
3.4. Misión con relevo en simulación. Cada ventana es un proceso de Gazebo con un piso y su vehículo; el láser de cada uno se ve en azul. La terminal muestra el resultado de la acción: éxito, un relevo y 100,1 s de duración. 36
3.5. Arquitectura del sistema en la campaña con los dos vehículos, en los pisos 3 y 4. La estructura es la de la figura 3.3; cambia la implementación de los bloques. 38
3.6. Tres recorridos en el hall del piso 2. El círculo es la pose que reporta la localización al terminar; el cuadrado, la posición real medida con flexómetro. La separación entre los dos es el error, y su causa está en la odometría, no en el algoritmo de localización. 42
3.7. Mapa de navegación del piso 4, derivado del levantamiento con flexómetro. El pasillo corre 25 m del extremo norte al sur; los salones quedan al este, y el ascensor y la escalera al oeste. 43
3.8. Interfaz durante una misión en simulación: el destino elegido, el botón de cancelar y el panel de estado del primer tramo, con el texto del coordinador, el vehículo activo y la distancia restante. 48
3.9. Interfaz en la etapa de espera de confirmación, durante su prueba automática: el estado de la misión lo publica la prueba, con el texto «Prueba de etapa 7», y la página muestra el botón «Ya estoy en el otro piso», de la versión anterior a los ajustes de la sección 3.5.5. 49

4.1. Una misión de la campaña en simulación. Se superponen el primer plan que calcula el navegador, la pose estimada a lo largo del recorrido y la odometría cruda. La separación creciente entre las dos últimas es la deriva que la localización corrige contra el mapa. 57

4.2. Cadena de tres tramos en el piso 4 y regreso a la salida: recorrido según la localización, llegadas medidas con flexómetro, la llegada estimada al Salón 403 y la pose de la localización al terminar cada tramo. 59
4.3. Duración de cada etapa en las nueve misiones de la campaña con los vehículos, según las marcas de etapa del coordinador. El tramo rayado es la espera de la confirmación de cambio de piso, contada desde que el segundo vehículo está en su escalera. 62

LISTA DE TABLAS

2.1. Líneas de antecedentes, lo que aportan a este trabajo y el supuesto que este trabajo cambia. 9

3.1. Etapas del trabajo frente a las fases del anteproyecto. 25
3.2. Pasos de la integración en simulación, en el orden en que se dieron. 29
3.3. Pasos del trabajo con los vehículos físicos, en el orden en que se dieron. 30
3.4. Etapas de una misión, tal como las publica el coordinador. 34
3.5. Cambios en la implementación al pasar de la simulación a los vehículos. 37
3.6. Componentes de cada vehículo y del computador central. 40
3.7. Definición operativa de las cuatro métricas del anteproyecto. 51
3.8. Pruebas de verificación de la demostración física y su estado al cerrar la toma de datos. 54

4.1. Tasa de éxito de la campaña en simulación, con intervalo de confianza de Wilson al 95 %. 55
4.2. Tiempo de respuesta de las treinta misiones, en escalones del reloj de la simulación. 56
4.3. Fracción del avance real que registra la odometría, por entorno. El criterio de la prueba es un error de 10 % o menos. 58
4.4. Cadena de tres tramos en el piso 4 y regreso a la salida: llegada de cada tramo. . . . 59
4.5. Campaña con los dos vehículos: las nueve misiones sorteadas que corrieron, todas completadas con un relevo. La llegada es la de la localización del vehículo, con una tolerancia de 0,5 m. 61

INTRODUCCIÓN

Orientarse dentro de un edificio grande no es igual de fácil para todo el mundo. Quien llega por primera vez construye un mapa mental a partir de información parcial, y lo va corrigiendo mientras camina. Ese esfuerzo crece cuando el espacio tiene pasillos repetidos, pocos puntos de referencia o señalización escasa. Y crece más cuando hay que cambiar de piso, porque la escalera corta la continuidad de lo que la persona lleva recorrido. Jiménez Muñoz lo señala al construir una arquitectura cognitiva que mete al usuario dentro de un sistema de mapas de varios niveles: la escalera es el punto donde la representación mental del recorrido deja de servir [1].

La robótica móvil colaborativa lleva años resolviendo tareas parecidas. El reparto de trabajo entre varios robots tiene base teórica desde que Gerkey y Matarić [2] formalizaron la asignación de tareas en sistemas multi-robot. Y Varghese et al. [3] muestran que varios agentes pueden mapear y localizarse de forma cooperativa. Casi todos esos sistemas operan, sin embargo, en una sola planta.

El cambio de nivel se ha abordado por dos caminos, y los dos pasan por el hardware. Kim et al. [4] combinan un mapa por piso con un grafo topológico y hacen que el robot use el ascensor. Song et al. [5] van más lejos: automatizan ese uso instalando dispositivos conectados en el edificio. La otra opción es dotar al robot de mecanismos para subir escaleras. Ambas encarecen la plataforma y exigen intervenir la infraestructura.

Este trabajo propone otra. En vez de un robot que cruza de piso, se usan **dos robots, cada uno dedicado a un nivel**. El usuario sube por su cuenta y un segundo agente lo recibe arriba. El cambio de piso deja de ser un problema de locomoción y pasa a ser uno de coordinación. Dos agentes se reparten una tarea y se la pasan en un punto acordado. A ese traspaso lo llamamos *relevo*, y es el aporte que el proyecto declara.

La pregunta que organiza el trabajo es si ese esquema funciona. No basta con que los robots se muevan. Hay que demostrar que la asistencia no se interrumpe cuando el usuario cambia de nivel, y medirlo con criterios fijados antes de correr los experimentos. Por eso el proyecto dedica tanto espacio al protocolo experimental como a la implementación.

El sistema se construyó sobre dos vehículos AWS DeepRacer y software libre. La coordinación corre en un nodo central que habla con cada agente a través de ROS 2. Macenski et al. [6] lo describen como un rediseño sobre DDS, no como una actualización de su antecesor. El usuario pide el servicio desde una página web que funciona en cualquier teléfono, sin instalar nada y sin salida a internet. La

evaluación se hizo primero en simulación, con una campaña de treinta misiones sorteadas, y después sobre los vehículos reales en los pasillos del edificio, con una campaña de diez misiones sorteadas entre los pisos 3 y 4.

El documento sigue ese orden. El capítulo 1 plantea el problema y los objetivos. El 2 reúne el marco conceptual, teórico, legal y ambiental. El 3 describe cómo se construyó el sistema: primero en el orden en que se hizo, desde la plataforma que proponía el anteproyecto, y después por objetivo específico. El 4 presenta los resultados y, con el mismo énfasis, sus límites. Los capítulos 5 y 6 cierran con las conclusiones y las líneas que quedan abiertas.

1. CONTEXTUALIZACIÓN

1.1 Planteamiento del problema

Orientarse en un edificio desconocido exige construir y actualizar una representación mental del espacio. El proceso se complica cuando el entorno tiene visibilidad limitada, estructuras repetidas y varios puntos de decisión. Omer y Monteriu [7] describen esos espacios compartidos con personas como el escenario que más exige a un robot. Vale también para quien camina sin conocerlos.

Las edificaciones de varios pisos añaden una dificultad propia. El cambio de nivel rompe la continuidad espacial y obliga a integrar información vertical en el recorrido. Es el mismo problema que Kim et al. [4] resuelven para un robot, con un grafo topológico entre plantas.

La robótica móvil colaborativa ha avanzado en exploración, logística y cobertura distribuida. La asignación de tareas entre agentes tiene una formalización aceptada desde el trabajo de Gerkey y Matarić [2]. Pero la mayoría de esos desarrollos se evaluó en entornos de una sola planta, donde la segmentación vertical no es el problema central. Los casos de limpieza doméstica y de almacén que documenta Nair [8] son representativos de ese escenario plano.

Resolver el cambio de piso con un solo robot resulta caro. Exige mecanismos de locomoción capaces de subir escaleras, o intervenir el sistema de control del ascensor. Song et al. [5] lo hacen instalando dispositivos conectados en la propia cabina. Las dos opciones elevan el costo del hardware y obligan a modificar el edificio.

De ahí el planteamiento de este trabajo. El problema no es cómo hacer que un robot suba, sino **cómo lograr que dos robots sencillos se repartan el recorrido y se pasen al usuario sin que la asistencia se interrumpa**. La comunicación entre agentes sustituye al hardware de transición vertical.

1.2 Pregunta problema

> ¿Es viable mantener la continuidad de la asistencia de orientación a una persona que cambia de nivel en un edificio, mediante un esquema de relevo entre dos robots móviles dedicados cada uno a una planta, sin que ninguno de ellos atraviese la zona de transición

vertical?

La pregunta tiene dos mitades y las dos se responden en el capítulo 4. La primera es de funcionamiento: si el relevo ocurre. La segunda es de medida: con qué incertidumbre se puede afirmar.

1.3 Objetivos

1.3.1 Objetivo general

Implementar un sistema colaborativo de dos robots móviles terrestres para la guía coordinada y eficiente de asistencia a visitantes en entornos interiores con múltiples pisos, mediante comunicación inter-robot, asignación dinámica de tareas y una interfaz móvil de interacción usuario-robot.

1.3.2 Objetivos específicos

1. Modelar la arquitectura funcional del sistema colaborativo de dos robots móviles, definiendo requerimientos, escenarios de operación con múltiples pisos, estrategia de asignación de tareas y esquema de comunicación inter-robot.
2. Desarrollar una plataforma robótica móvil basada en dos vehículos tipo DonkeyCar, integrando los módulos de locomoción, sensado, procesamiento y comunicación necesarios para la operación colaborativa en entornos interiores.
3. Programar una interfaz móvil para interacción humano-robot (HRI) basada en la selección de localizaciones de interés de origen y destino.
4. Evaluar el desempeño del sistema mediante pruebas experimentales en un entorno interior controlado, utilizando métricas como tiempo de respuesta, tiempo de asignación de robot, tasa de éxito en la entrega de asistencia y continuidad del servicio entre niveles.

Estos cuatro objetivos organizan el resto del documento. El capítulo 3 dedica una sección a cada uno, y el 4 reporta lo que se midió para cada uno.

2. MARCO REFERENCIAL

Este capítulo sitúa el trabajo frente a lo que ya existe y fija el vocabulario y la teoría con que se lee el resto del documento. Los antecedentes revisan cinco líneas de trabajo y terminan con lo que este proyecto aporta frente a ellas. El marco conceptual define los términos técnicos. El marco teórico expone el fundamento de cada decisión de diseño y la literatura que lo respalda. El marco legal y el ambiental sitúan el prototipo frente a la normativa que le aplica y frente a su impacto.

2.1 Antecedentes

La revisión se ordena en cinco líneas: robots que guían a personas, navegación autónoma en interiores, coordinación entre varios robots, movilidad entre pisos y distribución del cómputo. Cada una resuelve bien una parte del problema. La última subsección reúne lo que ninguna resuelve, que es lo que este trabajo aborda.

2.1.1 Robots que guían a personas

La idea de un robot que acompaña a una persona dentro de un edificio público tiene más de veinticinco años. Burgard et al. [9] describen RHINO, un robot guía que operó de forma autónoma durante seis días en un museo en 1997. Su arquitectura integraba localización, construcción de mapas, evasión de obstáculos, planificación e interacción con los visitantes, además de una interfaz web para seguirlo a distancia. De ese trabajo se toma una idea que sigue vigente: guiar personas es un problema de integración tanto como de navegación, porque ninguna de las piezas sirve si las demás fallan. Del mismo grupo salió poco después la localización de Monte Carlo [10], que es la base del algoritmo de localización que usan los vehículos de este proyecto.

Guerreiro et al. [11] trasladan la idea a la asistencia de personas ciegas. Su robot, CaBot, tiene forma de maleta y transmite la dirección a seguir por medio de un mango que vibra. Lo evaluaron con diez participantes ciegos, que lo compararon con un perro guía o con un acompañante vidente. El trabajo muestra dos exigencias que también aparecen aquí. El robot tiene que adaptarse a una persona que camina a su lado. Y la interfaz tiene que comunicar el estado del guiado sin exigir atención constante.

Compartir el espacio con personas plantea además un problema de seguridad. Li y Gurocak [12] lo abordan modificando el algoritmo Bug con sensores de proximidad, para reducir las colisiones entre un robot colaborativo y las personas que trabajan a su alrededor. En este proyecto la respuesta es más modesta y está en la operación: una persona acompaña a cada vehículo durante las maniobras, y el usuario puede cancelar la misión en cualquier momento.

Jiménez Muñoz [1] integra al usuario dentro del propio sistema de mapas, combinando representaciones semánticas con reconocimiento de personas para localizar y seguir a quien acompaña, en una plataforma asistencial de varios niveles. Es el antecedente más próximo en intención, porque trata a la persona como parte del problema de navegación y no como un destinatario externo.

En los tres casos un mismo robot acompaña a la persona durante todo el recorrido. Ese supuesto es el que este trabajo cambia.

2.1.2 Navegación autónoma en interiores

Sojan et al. [13] integran percepción, localización y planificación sobre ROS 2 para desplazamiento autónomo en interiores dinámicos. Su aporte es consolidar el flujo completo (mapa, localización por filtro de partículas y planificación) sobre una representación plana del espacio. Es la arquitectura que este proyecto toma como punto de partida en cada vehículo.

El sistema de navegación que implementa ese flujo en ROS 2 es Nav2. Macenski et al. [14] lo presentan como sucesor de la pila de navegación de la primera generación de ROS. Organiza la tarea con un árbol de comportamiento y gestiona el arranque de cada servidor con nodos de ciclo de vida, que deben configurarse y activarse antes de funcionar. Lo validaron en un campus universitario, con el robot navegando entre estudiantes durante una prueba de larga duración. Macenski et al. [15] recogen, desde la experiencia de los propios mantenedores, los algoritmos de planificación, control y localización que hoy ofrece el ecosistema, con pruebas comparativas entre ellos. Las dos referencias sirvieron para elegir los componentes que se describen en la sección 2.3.3.

Varghese et al. [3] llevan el problema a un escenario cooperativo: varios agentes construyen un mapa compartido y mantienen consistente su localización. El énfasis se desplaza de la autonomía individual a la construcción colectiva del entorno. Con el trabajo anterior comparte la técnica, porque las dos propuestas usan representaciones bidimensionales y se apoyan en ROS; la diferencia está en el alcance.

Existe una línea distinta que resuelve la navegación con aprendizaje en lugar de con planificación explícita. Bojarski et al. [16] mostraron que una red neuronal entrenada de extremo a extremo, solo

con imágenes de cámara y el ángulo de dirección del conductor, aprende a conducir sin que nadie programe el seguimiento de carril. Zain [17] aplica una idea parecida a la evasión de obstáculos, fusionando sensores con una red convolucional, y López Velásquez et al. [18] combinan aprendizaje profundo con lógica difusa para obtener autonomía reactiva.

Este proyecto no sigue esa vía. Entrenar cualquiera de esos modelos exige un conjunto de datos etiquetados del edificio concreto, que no existe y que habría que levantar. Además, el aporte declarado aquí está en la coordinación de dos agentes que se reparten un recorrido, y la percepción puede resolverse con métodos establecidos. Con planificación clásica cada decisión del vehículo se puede inspeccionar, y eso pesa cuando hay que explicar un fallo. Una red entrenada podría dar mejor desempeño reactivo, pero no permitiría explicar por qué tomó una decisión.

2.1.3 Coordinación entre varios robots

Las estrategias de coordinación de la literatura revisada se diferencian por el nivel en el que cooperan. Nair [8] funda la cooperación en el intercambio de información sensorial: los robots comparten lo que perciben para mejorar el desempeño del conjunto. Roszkowska et al. [19] proponen en cambio una estructura jerárquica que supervisa el desplazamiento simultáneo de varios agentes, con el fin de prevenir conflictos. Rudrusamy et al. [20] resuelven el problema desde el canal de comunicación, con un esquema de control por difusión y seguimiento visual.

Xu et al. [21] aportan una arquitectura modular sobre ROS para asignación cooperativa de tareas, y Zhan et al. [22] estudian la resiliencia del sistema cuando un nodo falla. El marco teórico de la asignación de tareas lo fijan Gerkey y Matarić [2], cuya taxonomía permite clasificar cualquier esquema de reparto según cuántas tareas acepta un robot, cuántos robots requiere una tarea y con cuánta anticipación se decide. La sección 2.3.1 usa esa taxonomía, y su extensión por Korsah et al. [23], para situar el esquema de este trabajo.

El antecedente más cercano a la propuesta de este proyecto es Srivastava et al. [24], que coordinan recogida y entrega entre varios robots mediante planificación con relevos. Allí lo que se traspasa de un agente a otro es un paquete; aquí es una persona. La diferencia tiene consecuencias de diseño: un paquete puede esperar indefinidamente en el punto de transferencia, y una persona no. Por eso el relevo de este trabajo incluye una confirmación explícita del usuario cuando llega al otro piso, con un aviso a los 60 s y un plazo máximo de 120 s.

Un antecedente de alcance comparable, por tratarse también de un proyecto de grado de ingeniería, es Piguave Toala y Ronquillo Manosalvas [25], que implementan un sistema multi-robot de código abierto sobre ROS 2 para ambientes colaborativos.

Según la revisión del anteproyecto, todos estos trabajos operan en un solo nivel físico, sin segmentación vertical.

2.1.4 Movilidad entre pisos

El cambio de nivel se ha abordado sobre todo desde la logística y la exploración. Kim et al. [4] resuelven la planificación tridimensional combinando un mapa por cada piso con un grafo topológico que los conecta, e integran además la interacción con el ascensor. Song et al. [5] se concentran en esto último: automatizar el uso del elevador mediante dispositivos conectados instalados en el edificio.

Luo et al. [26] plantean la pregunta un paso antes (ascensor o escalera) y la resuelven con aprendizaje por refuerzo jerárquico, de modo que el robot elige el medio de transición según el tránsito que encuentra. En otra dirección, Chen et al. [27] estructuran el entorno con una descomposición piso-escalera que extiende las técnicas bidimensionales a configuraciones de varios niveles, y Gong et al. [28] incorporan conciencia de piso a la navegación hacia objetos sin entrenamiento previo del escenario.

Las cuatro propuestas resuelven la movilidad entre pisos desde ángulos complementarios: la infraestructura física, el modelado topológico y la decisión adaptativa. Comparten un supuesto: el robot cruza de nivel, sea subiendo o sea accionando el ascensor. Y todas se orientan a entrega o exploración antes que a la asistencia de personas.

2.1.5 Distribución del cómputo

La última línea trata de dónde debe ejecutarse el procesamiento. Kehoe et al. [29] revisan las arquitecturas que lo delegan a servidores remotos, y Tahir y Parasuraman [30] las que lo acercan al borde de la red. Malavolta et al. [31] miden el costo real de esa delegación sobre WiFi en robots terrestres, que es el caso de este proyecto.

Sus resultados pesaron en una decisión concreta. La navegación corre completa a bordo de cada vehículo, y lo único que se centraliza es la coordinación, que intercambia mensajes pequeños y corre en un computador portátil. Además, la interfaz debe funcionar sin salida a internet, porque la red del edificio no siempre la tiene. La sección 2.3.2 explica cómo esa distribución se refleja en la red de comunicaciones.

2.1.6 Aporte de este trabajo frente a los antecedentes

La literatura cubre bien cada componente por separado. Hay flujos de navegación completos sobre ROS, esquemas de coordinación multirrobot a distintos niveles, soluciones de movilidad vertical y trabajo sobre la integración del usuario. La tabla 2.1 resume lo que cada línea aporta a este proyecto y el supuesto que este proyecto no comparte.

Tabla 2.1. Líneas de antecedentes, lo que aportan a este trabajo y el supuesto que este trabajo cambia.

Línea	Lo que se toma de ella	Supuesto que no se comparte
Robots guía	La integración de navegación e interacción; la localización de Monte Carlo	Un solo robot acompaña todo el recorrido
Navegación en interiores	El flujo mapa, localización y planificación sobre ROS 2 (Nav2)	Ninguno: se adopta, adaptado a la cinemática del vehículo
Coordinación multirrobot	La taxonomía de asignación; la planificación con relevos	Lo que se traspasa puede esperar sin límite
Movilidad entre pisos	El mapa por piso unido por un punto de transición	El robot cruza de nivel
Distribución del cómputo	La navegación a bordo y la coordinación en un nodo central	—

Lo que no aparece en ninguna línea es la combinación que aquí se propone: resolver la discontinuidad entre pisos sin que ningún robot la atraviese. El recorrido se reparte entre dos agentes dedicados, cada uno a un nivel, y la persona pasa de uno a otro en la zona de transición.

La diferencia con los trabajos de movilidad vertical es económica antes que algorítmica. Esos trabajos obtienen la continuidad con hardware: mecanismos de locomoción para escaleras o intervención del ascensor. Este proyecto la obtiene con comunicación entre agentes, que es lo que los trabajos de coordinación ya resuelven, aplicada a un problema donde no se había usado.

2.2 Marco conceptual

Aquí se definen los términos necesarios para leer el resto del documento. El lector supuesto tiene formación en ingeniería electrónica. Las seis categorías son las del anteproyecto, y dentro de cada una se añaden los términos que aparecieron durante la implementación. El fundamento teórico de

cada concepto está en la sección 2.3.

2.2.1 Navegación y percepción

Se llama **navegación autónoma** al desplazamiento de un robot sin intervención humana. Reúne tres capacidades que trabajan juntas: percibir, saber dónde se está y decidir hacia dónde moverse. La primera pieza de «saber dónde se está» es la **odometría**, que estima el desplazamiento del vehículo desde su punto de partida a partir de sus propias mediciones. Cuando esas mediciones salen de comparar barridos sucesivos de un sensor láser, se habla de **odometría láser**; es la única disponible en los vehículos de este proyecto, que no tienen codificadores en las ruedas. Toda odometría se desvía con el recorrido, así que hay que corregirla contra una referencia externa. De eso se encarga **AMCL**, un algoritmo de localización por filtro de partículas que combina la odometría con las lecturas del láser y sitúa al vehículo dentro de un mapa conocido [10].

El sensor láser de barrido se llama **LiDAR**: mide distancias en un plano, en 360° alrededor del vehículo. Junto a él, la tarjeta de cada vehículo incluye una **unidad de medición inercial** o IMU, con un giroscopio que mide la velocidad de giro y un acelerómetro. Las dos fuentes se combinan con un **filtro de Kalman extendido**, un estimador que pondera cada medición según su incertidumbre y entrega una sola estimación del movimiento [32].

Hay un concepto que decide si todo lo anterior puede funcionar: la **observabilidad** de un estado. Una magnitud es observable cuando se puede estimar a partir de las mediciones disponibles. La propiedad depende de la combinación de entorno y sensor, no solo del sensor, de modo que una misma magnitud puede ser observable en un sitio e inobservable en otro. Cuando es inobservable, ningún ajuste de parámetros la recupera, porque la información no está en la señal. Para cuantificarla, el proyecto definió la **información de avance** de un barrido: la fracción de los rayos del LiDAR que inciden sobre superficies orientadas hacia la dirección de marcha. Solo esos rayos cambian de longitud cuando el vehículo avanza. La sección 2.3.5 desarrolla el caso.

El entorno se representa de dos formas complementarias. El **mapa de ocupación** dice qué celdas están libres y cuáles no. El **mapa de costos** asigna a cada celda un costo de tránsito en lugar de una sola etiqueta, de modo que el planificador puede preferir el centro del pasillo sin que los bordes queden prohibidos [33]. Hay un mapa de costos global, que cubre todo el mapa, y uno local, limitado al entorno inmediato del vehículo, que registra los obstáculos que el láser ve en ese momento.

Sobre esa representación se calcula la trayectoria. En un vehículo que gira con radio mínimo y puede ir hacia adelante y hacia atrás, las trayectorias más cortas son las del **modelo de movimiento Reeds-Shepp** [34]. El punto donde el vehículo invierte el sentido de la marcha se llama **cúspide**, y

contarlas indica cuántas maniobras hicieron falta. Al final del recorrido interviene el **verificador de meta**, que decide si el vehículo llegó comparando su posición, y a veces su orientación, contra una tolerancia.

Queda un término que es del edificio y no del vehículo: la **zona de transición**, el lugar donde se cambia de nivel. En este sistema es la escalera, y ningún vehículo la atraviesa.

2.2.2 Control y locomoción

La **cinemática de robots móviles** es el modelo matemático que relaciona las velocidades de los actuadores con el movimiento del vehículo [35]. La plataforma de este trabajo sigue un **modelo cinemático tipo automóvil**, también llamado **dirección Ackermann**: cuatro ruedas, dirección en el eje delantero y una relación entre el ángulo del servomotor y el radio de giro. Ese modelo hace que las ruedas interior y exterior giren en torno al mismo centro instantáneo de rotación, de modo que no derrapan. Lo que impone, y condiciona todo lo demás, es un **radio de giro mínimo**: el vehículo no puede girar sobre su propio eje. Un robot que sí puede hacerlo, como uno de tracción diferencial, se dice que gira en el sitio.

Esa restricción obliga a elegir con cuidado los dos componentes que calculan y siguen la trayectoria. La planificación corre a cargo de un **planificador híbrido A***, que busca rutas trabajando sobre posición y orientación a la vez [36]. El seguimiento lo hace un **controlador de persecución pura regulada**, que persigue un punto móvil sobre la trayectoria y regula la velocidad según la curvatura y la cercanía de obstáculos [37].

Las órdenes llegan a los actuadores como señales de **PWM**, modulación por ancho de pulso, que controlan el motor de tracción y el servomotor de dirección variando el ciclo de trabajo. En vehículos con realimentación de velocidad, la regulación fina se hace con un **control PID**, proporcional-integral-derivativo. El vehículo de este proyecto no tiene esa realimentación: su tracción funciona en lazo abierto, y la misma orden produce velocidades distintas según la carga de la batería. Entre la orden y el movimiento aparece además la **banda muerta de tracción**, el rango de velocidades pedidas en el que el motor no recibe potencia suficiente para mover el vehículo. En esta plataforma está por debajo de 0,40 m/s.

Por último, la **media vuelta en varios tiempos** es la maniobra con que un vehículo de este tipo invierte su rumbo en un pasillo angosto: alterna marcha atrás y avance con la dirección a tope, hasta completar el giro.

2.2.3 Comunicación y sistemas distribuidos

Un **middleware robótico** es la capa de software que comunica entre sí los módulos del sistema. Aquí es **ROS 2**, que organiza el software en **nodos**, programas que cumplen cada uno una función, y los comunica por **tópicos** de publicación y suscripción, **servicios** de petición y respuesta, y **acciones** [6]. Por debajo, ROS 2 implementa todo eso sobre **DDS**, un estándar de comunicación en el que los participantes de la red se descubren solos [38].

De ese mecanismo de **descubrimiento** salen varios conceptos que en la práctica deciden quién habla con quién. El **dominio** es un número que segmenta el descubrimiento: dos procesos con dominio distinto no se ven entre sí. La **partición de comunicaciones** es un filtro más fino, dentro de un mismo dominio, que solo deja intercambiar mensajes a los procesos de la misma partición. El **espacio de nombres** es un prefijo, como `/robot1`, que se antepone a los tópicos, servicios y acciones de un nodo para distinguir a qué vehículo pertenecen. Y el **rango de descubrimiento** limita a quién se anuncia cada proceso: solo a los de su propio computador, o también a una lista fija de equipos. Con esa última opción la red adopta una **topología en estrella**, en la que los vehículos se comunican con el computador central y no entre sí.

De los patrones de comunicación, el que sostiene el sistema es la **acción**, pensada para tareas largas. Tiene una meta, retroalimentación periódica y un resultado final, y se puede cancelar. El nodo que la ofrece es el **servidor de acción** y el que la invoca, el **cliente**. El coordinador de este proyecto cumple los dos papeles: es cliente ante cada agente y servidor ante la interfaz. Para aceptar cancelaciones, un servidor de acción debe registrar una función de respuesta a la cancelación; si no lo hace, ROS 2 las rechaza todas sin informar del motivo. Cada canal lleva además sus políticas de entrega, que se llaman **calidad de servicio** y se configuran por separado.

La interfaz web no habla ROS, así que entre las dos hay un traductor: el **puente *rosbridge***, un servidor que convierte entre el protocolo de ROS 2 y mensajes JSON sobre WebSocket [39]. Es lo que permite que una página web opere el sistema sin instalar nada. Finalmente, un **contenedor** es un entorno de ejecución aislado, con su propio sistema de archivos y sus propias bibliotecas, que corre sobre el sistema operativo del equipo. Permite tener en un mismo computador dos versiones distintas de ROS 2.

2.2.4 Arquitectura del sistema

La **asignación de tareas** es el reparto de actividades entre los robots según su disponibilidad y su ubicación [2]. A veces la tarea no cabe en un solo agente, porque atraviesa la frontera entre dos nive-

les. Entonces el reparto toma la forma de un **protocolo de relevo**: el traspaso de la responsabilidad de guiar al usuario de un agente a otro en un punto definido. Ese lugar, que en este edificio es la escalera, se llama **punto de transferencia**.

Tres piezas más completan la arquitectura. La **máquina de estados de misión** es un conjunto ordenado de estados que describe en qué punto va una misión; se publican para que la interfaz y el registro de evaluación lo sepan. El **planificador puro** calcula los tramos de la misión a partir del catálogo de puntos y del par origen-destino. Se llama «puro» porque no depende del middleware ni de ningún estado externo, lo que permite verificarlo con cientos de casos en milisegundos, sin levantar el simulador. Y el **registro de misión** es un documento estructurado que se compone a partir de la grabación de cada ejecución y se valida contra un esquema. Es la evidencia primaria del experimento.

2.2.5 Interacción humano-robot

La **interacción humano-robot**, o HRI por sus siglas en inglés, es el campo que estudia el diseño y la evaluación de sistemas robóticos pensados para interactuar con personas [1]. Su parte visible aquí es una **interfaz gráfica de usuario**, el entorno visual donde la persona y la máquina se encuentran. El acto que la pone en marcha es la **solicitud de servicio**: la petición del usuario para recibir guiado.

Dos decisiones de diseño tienen nombre propio porque se tomaron después de ver el sistema en uso. La **interfaz responsiva de selección** permite buscar un destino escribiendo su nombre, sin perder la opción de ver el catálogo completo. Con diez destinos la diferencia es pequeña; con cien no lo sería. Y la **retroalimentación literal** es un principio: el texto que ve el usuario lo redacta el componente que conoce el estado del sistema, y la interfaz lo muestra tal cual. Así la interfaz no puede contradecir al coordinador.

2.2.6 Métricas y criterios de evaluación

Las cuatro métricas del proyecto son las del anteproyecto. El **tiempo de respuesta** es lo que pasa entre la solicitud del usuario y el momento en que el vehículo arranca la tarea. El **tiempo de asignación** es lo que pasa entre aceptar la solicitud y designar qué agente la atiende. La **tasa de éxito** es la proporción de solicitudes que terminan bien. Y la **continuidad de servicio** es la capacidad de mantener la asistencia mientras el usuario cambia de nivel.

Medir bien exige cuatro conceptos más, que el proyecto incorporó durante la evaluación. El **intervalo de confianza de Wilson** estima el intervalo de una proporción y es más fiable que la aproximación

normal cuando la muestra es pequeña o la proporción se acerca a 0 o a 1 [40]. La **falsabilidad** de un criterio es la propiedad de que alguna ejecución del sistema pueda, al menos en principio, incumplirlo [41]. Un **invariante estructural** es una propiedad que se cumple siempre por el diseño del sistema, de modo que no es un resultado experimental y no debe presentarse como tal. Y la **cota superior por misión** resuelve qué hacer cuando el instrumento no tiene resolución para dar el valor exacto: se informa el límite que sí se puede garantizar.

La demostración sobre los vehículos reales se organizó en **pruebas de verificación**: condiciones comprobables, acordadas con los directores antes de correrlas, que había que superar en orden para pasar a la siguiente etapa. Un ejemplo es que la odometría registre el avance con un error del 10 % o menos. Para decidir si se superaron hace falta una **verdad de terreno**, la medida independiente con la que se contrasta el sistema, que en este trabajo es la distancia tomada con flexómetro. En la campaña con los dos vehículos esa medida faltó, y la sección 4.4.4 explica qué cambia en la lectura de sus resultados.

2.3 Marco teórico

Cada subsección expone el fundamento de una parte del sistema y la literatura que lo respalda. El orden va de la coordinación, que es el aporte del trabajo, a la navegación de cada vehículo, y termina en los criterios con que se evaluó.

2.3.1 Asignación de tareas en sistemas multirrobot

Gerkey y Matarić [2] clasifican los problemas de asignación de tareas entre robots según tres ejes. El primero distingue robots que ejecutan una sola tarea a la vez (ST, *single-task*) de robots que atienden varias (MT). El segundo distingue tareas que requieren un solo robot (SR, *single-robot*) de tareas que requieren varios (MR). El tercero distingue la asignación instantánea (IA), que se decide al llegar cada tarea, de la extendida en el tiempo (TA), que planifica asignaciones futuras.

La asignación de este sistema es ST–SR–IA. Cada vehículo ejecuta un tramo a la vez, cada tramo lo ejecuta un solo vehículo, y el agente se elige al llegar la solicitud, sin planificar las siguientes. Los mismos autores demuestran que esa clase es una instancia del problema de asignación óptima, que el método húngaro resuelve en tiempo polinómico [42]. Es decir, el óptimo está disponible y no es costoso de calcular.

En este problema, sin embargo, la matriz de costos tiene un solo candidato admisible por tarea. Cada

vehículo está confinado a su nivel y hay uno solo por nivel. Aplicar el método húngaro a esa matriz devuelve siempre la misma correspondencia que el sistema guarda como tabla de configuración, en la que cada nivel tiene asignado su vehículo. La ausencia de un optimizador es, por tanto, una consecuencia de la estructura del problema y no una carencia de la implementación.

Por la misma razón se descartaron los mecanismos de mercado. En el protocolo de red de contratos [43], un gestor anuncia la tarea, los agentes envían una oferta con su costo y la tarea se adjudica a la mejor. MURDOCH [44] aplica ese esquema a robots sobre un modelo de publicación y suscripción, y CBBA [45] lo lleva a una subasta descentralizada en la que los agentes llegan a consenso sin un coordinador central. Con un solo agente elegible por tramo, cualquier subasta tiene un único postor: añade mensajes y latencia sin cambiar la decisión. CBBA, además, elimina el coordinador central, que la arquitectura mantiene por diseño.

La misión completa sí se sale de esa clase. El relevo impone una restricción de precedencia: el tramo del segundo vehículo no puede empezar hasta que el primero termine el suyo y el usuario cambie de piso. Las agendas de los dos agentes quedan acopladas, y la taxonomía de 2004 está construida para tareas independientes. Korsah et al. [23] la extienden según el grado de interdependencia entre tareas, y en su clasificación esta misión pertenece a la clase XD, de dependencias entre agendas (*cross-schedule dependencies*). Ahí está el aporte del trabajo: la asignación es trivial y óptima por construcción, y la coordinación del relevo pertenece a una clase de problema que la taxonomía clásica del área no contempla.

El análisis también delimita el alcance. El sistema atiende una misión a la vez y no tiene noción de agente ocupado ni cola de solicitudes. En cuanto haya dos o más vehículos elegibles por tarea, o misiones simultáneas, la matriz de costos deja de tener un único candidato y los métodos anteriores pasan a decidir de verdad. Esa extensión se plantea en los trabajos futuros.

2.3.2 ROS 2 y la comunicación entre los componentes

Todos los componentes se comunican mediante ROS 2. Macenski et al. [6] describen su arquitectura como un rediseño sobre DDS, el estándar de comunicación de datos en tiempo real del Object Management Group [38], y no como una actualización de su antecesor. DDS aporta el descubrimiento automático de los participantes y las políticas de calidad de servicio de cada canal.

El descubrimiento automático tiene un costo que crece con el tamaño del sistema. Por omisión, cada proceso se anuncia en la red y empareja sus publicadores y suscriptores con los de todos los demás. Con las dos cadenas de navegación completas encendidas, unos cuarenta procesos se anunciaban unos a otros por la red inalámbrica, y la carga de la tarjeta de un vehículo pasó de 6 a entre 17

y 33 (sección 3.4.9). ROS 2 Jazzy permite restringir el descubrimiento a los procesos del mismo computador y declarar, además, una lista fija de equipos con los que sí se comunica. Con esas dos opciones la red adopta la topología en estrella del diseño: dentro de cada vehículo los procesos se descubren entre sí, y hacia afuera solo se anuncia al computador portátil lo que el coordinador necesita. Los vehículos no se ven entre sí.

Dos mecanismos más separan a los vehículos. La partición de comunicaciones deja dentro de cada vehículo los tópicos del software del fabricante, que no llevan espacio de nombres. Y un espacio de nombres distinto para cada vehículo distingue la navegación, el estado y la maniobra de media vuelta de cada agente cuando llegan al portátil.

El sistema usa además dos distribuciones de ROS 2. La simulación corre sobre Humble, porque los paquetes que integran ROS 2 con la versión de Gazebo empleada no están publicados para Jazzy. Los vehículos corren sobre Jazzy, la que corresponde al sistema operativo de sus tarjetas, Ubuntu 24.04. El código fuente es el mismo en los dos destinos. Lo que cambió entre distribuciones es la definición del mensaje de la acción de navegación, de modo que un coordinador compilado en Humble no puede enviar metas a un agente compilado en Jazzy. Por eso el coordinador corre siempre en la distribución de los agentes que comanda: en simulación, sobre Humble; con los vehículos, sobre Jazzy, dentro de un contenedor en el portátil. La configuración de Nav2 tiene una variante por distribución, derivada de una base común.

2.3.3 Cinemática tipo automóvil y planificación de trayectorias

Un vehículo con dirección Ackermann está sujeto a una restricción no holonómica: no puede desplazarse de lado ni girar sobre su propio eje, y su curvatura está acotada por el radio de giro mínimo [35]. La consecuencia práctica es que no toda trayectoria libre de obstáculos es ejecutable. Un planificador que solo considera la posición propone giros que el vehículo no puede hacer.

Reeds y Shepp [34] resolvieron el problema geométrico de fondo: el camino más corto entre dos poses para un automóvil que puede avanzar y retroceder, con radio de giro mínimo. Demostraron que basta considerar caminos formados por arcos de radio mínimo y segmentos rectos, con a lo sumo dos inversiones de marcha, y que el óptimo está siempre entre un conjunto finito de candidatos calculables con fórmulas explícitas. Como el vehículo de este trabajo puede retroceder, ese es su modelo de movimiento.

Sobre un mapa con obstáculos hace falta además una búsqueda. Dolgov et al. [36] propusieron el A* híbrido: una búsqueda A* sobre el estado continuo del vehículo (posición y orientación), en la que cada expansión respeta la cinemática y el camino resultante se suaviza después. Lo validaron en los

estacionamientos del DARPA Urban Challenge de 2007, un escenario de maniobras con inversión de marcha parecido al de un pasillo angosto. Nav2 lo implementa en el planificador Smac, que admite el modelo Reeds-Shepp y está pensado para robots tipo automóvil [46]. Es el planificador que usan los vehículos de este proyecto.

El planificador trabaja sobre mapas de costos. Lu et al. [33] propusieron organizarlos por capas, cada una con una fuente de información. La capa estática viene del mapa y la de obstáculos, de las lecturas del sensor. La de inflado extiende el costo alrededor de cada obstáculo según el tamaño del vehículo. El mapa de costos global combina las tres y sirve al planificador; el local cubre los alrededores del vehículo y sirve al controlador. La configuración de este trabajo usa esas tres capas.

2.3.4 Seguimiento de la trayectoria

Una vez calculada la trayectoria, el controlador la convierte en órdenes de velocidad. La persecución pura [47] es un controlador geométrico. Elige sobre la trayectoria un punto situado a una distancia de anticipación ℓ del vehículo, y calcula el arco de circunferencia que lleva al vehículo hasta él. Llamando y a la distancia lateral del punto al eje del vehículo, la curvatura ordenada es

$$\kappa = \frac{2y}{\ell^2}. \tag{2.1}$$

Es simple, no requiere un modelo dinámico y se comporta bien en vehículos tipo automóvil, porque siempre ordena arcos que la dirección puede seguir.

Macenski et al. [37] le añadieron una regulación de velocidad pensada para robots de servicio. El controlador reduce la velocidad en las curvas cerradas y cerca de los obstáculos. Además admite la marcha atrás, de modo que sigue las cúspides que propone el planificador. En este trabajo se configuró con una velocidad deseada de 0,5 m/s, una distancia de anticipación entre 0,3 y 0,9 m, marcha atrás habilitada y el giro en el sitio deshabilitado, porque el vehículo no puede hacerlo.

El desarrollo permitió caracterizar un límite de esa configuración. Cuando una cúspide queda muy cerca de la meta, la distancia de anticipación se acorta, y por debajo de cierto umbral geométrico una guarda de seguridad del propio controlador anula la curvatura calculada. El vehículo sigue invirtiendo la marcha, pero ya no dirige, y maniobra decenas de veces en el mismo punto. El origen no está en el planificador ni en la coordinación, sino en un controlador de referencia que trabaja en el borde de su dominio de validez. Por eso la media vuelta se resolvió con una maniobra propia del agente (sección 3.4.10).

La banda muerta de tracción agrega otro límite. Como el vehículo no se mueve por debajo de

0,40 m/s, la velocidad mínima de aproximación del controlador se fijó en ese valor, y el vehículo llega a la meta sin una fase de aproximación lenta (sección 3.4.7).

2.3.5 Localización, odometría y observabilidad

La localización de Monte Carlo representa la creencia sobre la pose del vehículo con un conjunto de partículas, cada una con un peso [10], [32]. En cada paso, el modelo de movimiento desplaza las partículas según la odometría. Después, el modelo de medición las pondera según cuánto se parece el barrido del láser a lo que se vería desde esa pose en el mapa. Por último, un remuestreo concentra las partículas en las poses más probables. AMCL es la variante que ajusta el número de partículas a la incertidumbre; en este trabajo, entre 500 y 2000. Su precisión depende de la calidad de la odometría entre una actualización y la siguiente.

Como los vehículos no tienen codificadores en las ruedas, la odometría sale del láser. Jaimez et al. [48] estiman el movimiento plano de un láser a partir de barridos consecutivos con un enfoque de flujo de distancias (*range flow*). Alinean los barridos de forma densa, usando sus gradientes, sin buscar correspondencias entre puntos, y tardan alrededor de un milisegundo por barrido. Es el método que implementa el paquete rf2o, que usan los vehículos.

Todo estimador que se apoya en la geometría del entorno hereda una limitación. Zhang et al. [49] la formalizan como degeneración: hay direcciones del espacio de estados en las que las mediciones no restringen la solución, y en ellas el estimador no tiene información. Un pasillo recto y de sección uniforme es el caso típico. Al avanzar a lo largo de su eje, cada barrido es casi idéntico al anterior, y el estimador devuelve un desplazamiento cercano a cero. El resultado es coherente con lo que mide el sensor, porque desde él no ha cambiado nada. La posición longitudinal es inobservable, y ningún ajuste de parámetros la recupera.

De ahí sale la diferencia entre dos tipos de error que se parecen. Un error de calibración se corrige ajustando parámetros. Una inobservabilidad estructural no se corrige así, porque la información no está en la señal. La información de avance, definida en la sección 2.2, mide cuánta hay en cada sitio. El criterio que el proyecto adoptó para elegir dónde operar es que el entorno ofrezca estructura no uniforme dentro del alcance del sensor durante todo el recorrido.

La construcción de mapas comparte el mismo límite. La herramienta de SLAM de ROS 2 [50] construye el mapa a partir de la misma odometría del láser, y en un pasillo uniforme no tiene con qué cerrarlo. Por eso los mapas de este proyecto se derivaron de un levantamiento con flexómetro (sección 3.4.6).

El rumbo tiene una solución distinta, porque el giroscopio de la IMU lo mide de forma directa, sin depender del entorno. Un filtro de Kalman extendido combina las dos fuentes [32]. El paquete robot_localization lo implementa para ROS de forma general, permitiendo elegir qué componente de cada sensor se fusiona [51]. En este trabajo el filtro toma de la odometría láser las velocidades de avance y de la IMU la velocidad de giro. El rumbo mejora; la posición no, porque sigue saliendo del láser.

2.3.6 Simulación del sistema

La campaña de evaluación corrió en Gazebo, un simulador de código abierto pensado desde su origen para varios robots, con motor de física, modelos de sensores e integración con ROS [52]. Se usó la versión 11 de Gazebo, integrada con ROS 2 Humble. La cinemática Ackermann del vehículo se simula con un complemento propio del paquete de simulación del DeepRacer, en lugar del genérico de Gazebo.

La simulación levanta un proceso de simulación por vehículo y un solo dominio de comunicaciones para los dos. Los agentes se separan por espacio de nombres, incluido el marco de referencia de su mapa. Dar a cada vehículo un dominio distinto parece más simple, pero con la configuración por omisión corta el descubrimiento entre el coordinador y los agentes: separar por dominios y comunicar a los agentes con el coordinador resultan mutuamente excluyentes.

El tiempo que ven los nodos durante la simulación avanza en saltos de 100 ms, y eso fija la resolución con que se pueden medir los tiempos de la campaña. El efecto sobre las métricas se discute en la sección 4.4.

2.3.7 La interfaz web como cliente del sistema

Crick et al. [39] propusieron *rosbridge* para que programas ajenos a ROS, como un navegador, puedan usar tópicos, servicios y acciones a través de mensajes JSON sobre WebSocket. La versión para ROS 2 del puente expone las acciones con operaciones propias para enviar la meta, recibir la retroalimentación y recibir el resultado. La biblioteca de cliente más difundida del ecosistema web de ROS implementa las acciones de la primera generación de ROS, que no son compatibles con esas operaciones. Por eso la interfaz de este trabajo es un cliente propio, escrito sobre el protocolo del puente. Tampoco usa dependencias externas, porque debe funcionar sin salida a internet.

Un segundo principio rige la interfaz: no contiene lógica del dominio. El texto que lee el usuario en cada estado lo escribe el coordinador, y la interfaz lo muestra literal. Es la separación de

responsabilidades entre el dominio y la presentación. Si el navegador interpretara el estado para decidir qué texto mostrar, se duplicaría en el cliente una lógica que ya vive en el planificador, y las dos copias acabarían divergiendo.

2.3.8 Evaluación experimental

La evaluación se apoya en las cuatro métricas del anteproyecto, cada una con una definición operativa que indica qué marca del registro se resta de cuál. El protocolo se escribió antes de instrumentar el sistema, y las misiones se sortean con una semilla fijada antes de correrlas. Las dos decisiones tienen el mismo propósito: que ni el instrumento ni los recorridos se elijan después de ver los resultados.

La tasa de éxito es una proporción estimada sobre una muestra pequeña, y para ella el intervalo de confianza habitual, el de la aproximación normal, falla. Brown et al. [40] mostraron que su cobertura real oscila de forma irregular, incluso con muestras grandes, y empeora con proporciones cercanas a 0 o a 1. Recomiendan el intervalo de Wilson [53] para muestras pequeñas. Sea n el número de misiones, $\hat{p}$ la proporción observada de éxitos y z el cuantil de la confianza deseada ($z = 1{,}96$ para el 95 %). El intervalo de Wilson es

$$\frac{\hat{p} + \dfrac{z^2}{2n} \pm z\sqrt{\dfrac{\hat{p}(1-\hat{p})}{n} + \dfrac{z^2}{4n^2}}}{1 + \dfrac{z^2}{n}}. \tag{2.2}$$

Con las treinta misiones de la campaña y veintiséis éxitos, da entre 70,3 % y 94,7 %. El ancho del intervalo decide qué se puede afirmar: con esa muestra se sostiene que la tasa de éxito supera el 70 %, no que valga 86,7 %.

El segundo principio es de método. Para Popper [41], una afirmación aporta conocimiento empírico solo si alguna observación posible podría refutarla. Trasladado a un experimento, un criterio de aceptación solo es evidencia si alguna ejecución del sistema puede incumplirlo. En este trabajo hubo un caso concreto. El criterio de la variable de respuesta principal, la continuidad del servicio, estaba definido de forma que ninguna ejecución podía incumplirlo: los únicos puntos del código que producen un incumplimiento ocurren fuera de la ventana en que el criterio se evalúa. Treinta verificaciones satisfactorias de un criterio así no muestran que el sistema cumpla la propiedad. La corrección no fue inventar una ruta de fallo, que habría sido instrumentar el experimento para que pareciera más informativo. Se reformuló la afirmación como un invariante estructural del diseño y se respaldó con una magnitud que sí varía: el intervalo de tiempo entre el final del tramo de un vehículo y el inicio del otro (sección 4.4).

La demostración sobre los vehículos reales sigue otra lógica. La norma ASTM F3244-21 [54] establece un método de prueba para la navegación de robots en un área definida, con repeticiones del recorrido entre marcas fijas en el piso. El número de repeticiones fija cuánto se puede afirmar. Si las n repeticiones salen bien, la fiabilidad mínima que se demuestra con una confianza C es

$$R = (1 - C)^{1/n}, \tag{2.3}$$

que con $n = 10$ y $C = 0{,}85$ da $R \approx 0{,}83$. Con las cinco a diez repeticiones que caben en el calendario, la campaña física demuestra que el sistema opera sobre los vehículos, pero no estima su tasa de éxito con la precisión de la simulación. Por eso las dos campañas se reportan por separado: la de simulación produce la evidencia estadística y la física, la demostración. En los vehículos el protocolo mide la llegada con flexómetro y la tolerancia es de 0,5 m, fijada por el director para las pruebas con los vehículos; la simulación conserva 0,25 m.

2.4 Marco legal

El proyecto se desarrolla dentro de la política pública colombiana de transformación digital y de ciencia, tecnología e innovación, que enmarcan dos documentos. El CONPES 3975 [55] fija la Política Nacional para la Transformación Digital e Inteligencia Artificial. El CONPES 4069 [56] establece la Política Nacional de Ciencia, Tecnología e Innovación. Ambos piden a la academia que desarrolle prototipos con herramientas de la Cuarta Revolución Industrial, entre ellas la robótica autónoma, y que lo haga con diseño local e impacto social.

Este trabajo encaja en esa línea. Es un prototipo de asistencia robótica construido sobre plataformas de bajo costo y software abierto, dentro de las limitaciones presupuestales de un entorno universitario.

La presentación del propio documento sigue la norma NTC 1486 del ICONTEC [57], que regula cómo se presentan las tesis y los trabajos de grado y que la plantilla institucional declara como su base.

2.4.1 Alcance de la operación y responsabilidad

El alcance del proyecto excluye de forma explícita que los robots interactúen mecánicamente con el edificio. No accionan puertas ni botones de ascensor, no transportan carga y no cruzan las zonas de transición vertical. Esa delimitación se adoptó por razones técnicas y de costo, y tiene además una consecuencia normativa favorable: evita intervenir sistemas sujetos a regulación específica de

seguridad en edificaciones.

Las pruebas se hicieron en pasillos de uso común, en franjas de baja circulación, con una persona junto a cada vehículo durante las maniobras y la posibilidad de cancelar la misión desde la interfaz en cualquier momento.

2.5 Marco ambiental

El impacto ambiental del prototipo se concentra en tres frentes: el hardware que emplea, la energía que consume y el destino de sus componentes al final de la vida útil.

2.5.1 Reutilización de hardware existente

Las dos plataformas robóticas son equipo institucional que ya existía. El proyecto no las compró, y el presupuesto del anteproyecto las contabiliza como uso y desgaste. La decisión se tomó por razones económicas, pero reduce también la huella material del trabajo: no entran plataformas nuevas al inventario y se alarga la vida útil de equipos que estaban subutilizados.

Lo mismo ocurre con el entorno de evaluación. Es el propio edificio de la universidad, sin construir infraestructura dedicada ni modificar la existente. El sistema opera sobre la planta tal como está.

2.5.2 Consumo energético y autonomía

Cada vehículo lleva dos fuentes de alimentación independientes, una para el cómputo y otra para la tracción. Un tramo de navegación dura entre unos segundos y un minuto, y una sesión de campo completa gasta una fracción de la carga disponible.

El consumo de los vehículos durante toda la campaña es pequeño frente al de la estación de trabajo que ejecuta la simulación, que corre dos instancias del simulador a la vez y de forma sostenida. En un proyecto de robótica de esta escala, el mayor consumo energético está en la simulación y no en los robots. Ese hecho respalda la estrategia de verificación que se adoptó, en la que la lógica de coordinación se comprueba como función pura, sin simulador, en milisegundos y con un consumo despreciable.

2.5.3 Ciclo de vida de las baterías

Las baterías de polímero de litio que alimentan la tracción son el componente más sensible del sistema desde el punto de vista ambiental. Contienen litio y se degradan con los ciclos de carga. Al final de su vida útil deben ir a los canales de disposición de residuos electrónicos de la institución, nunca a la basura doméstica.

El nivel de carga influye además en el comportamiento del vehículo. Como la tracción funciona en lazo abierto, una misma orden produjo velocidades distintas en corridas consecutivas, y la batería es la causa más probable. Esa dependencia obliga a revisar el nivel de carga entre mediciones. Es una buena práctica experimental que además desincentiva el uso de baterías degradadas.

3. DESARROLLO

El sistema se construyó en cuatro frentes, uno por objetivo específico. Antes de describirlos, la primera sección ubica el trabajo en las fases de la metodología del anteproyecto. La segunda lo recorre en el orden en que se hizo, desde la plataforma DonkeyCar que proponía el anteproyecto hasta la campaña con los vehículos físicos, pasando por el modelo de AWS DeepRacer, su migración a ROS 2 Humble y la integración en simulación. Varias decisiones solo se entienden en ese orden. Las secciones siguientes, una por objetivo, describen qué quedó, qué se decidió y por qué. Las decisiones que hubo que revisar aparecen con su corrección, porque el camino que se descartó explica el que quedó.

3.1 Etapas del trabajo

El anteproyecto organizó el trabajo en seis fases: formulación, levantamiento de requisitos, diseño conceptual, desarrollo progresivo, integración y pruebas, y validación con análisis de resultados. El proyecto se ejecutó en dos semestres de 2026. En el primero, de febrero a junio, se cerraron las tres primeras fases y se empezó el desarrollo. En el segundo, de agosto a noviembre, se hicieron la integración y la evaluación, con un cronograma replanificado que aprobó el director al iniciar el semestre. La tabla 3.1 resume qué produjo cada período.

Tabla 3.1. Etapas del trabajo frente a las fases del anteproyecto.

Período	Fase	Lo que se hizo
Febrero y marzo	Planteamiento y requisitos	Anteproyecto, con el problema, la pregunta, la revisión de literatura y su sustentación; requisitos funcionales y restricciones técnicas
Abril	Requisitos y diseño conceptual	Análisis de la plataforma DonkeyCar y de los simuladores; adopción del modelo de AWS DeepRacer y su migración a ROS 2 Humble; arquitectura con servidor central y escenario de relevo
Mayo y junio	Desarrollo progresivo	Pasillo de la universidad en Gazebo; seguidor de paredes, descartado; sensores simulados en el primer piso; primer mapa con SLAM
Agosto	Integración	Vehículo físico con el software abierto de la comunidad; Nav2 en Humble para dirección Ackermann; contrato de interfaces; dos agentes en simulación; mundo de dos pisos; coordinador y relevo; requisitos numerados y protocolo experimental
Septiembre	Integración y validación	Campaña de treinta misiones en simulación; interfaz web en un teléfono; segundo vehículo; congelación del código; primera navegación autónoma sobre un vehículo
Octubre	Validación	Pisos 3 y 4; unidad inercial; llegada verificada; coordinador en el portátil; campaña con los dos vehículos

Dos documentos rigieron la segunda mitad. La matriz de requisitos asigna a cada requisito una prueba que lo declara cumplido o no, y es la que se usa para medir el avance de cada objetivo. Y los criterios de la demostración física, propuestos por los autores y decididos por los directores, fijan qué pruebas debían superarse sobre los vehículos reales y hasta cuándo se tomaban datos (sección 3.6.4).

3.2 Recorrido del trabajo

Esta sección cuenta el trabajo en el orden en que se hizo, con las decisiones que cambiaron su rumbo. Lo que quedó de cada etapa se describe con detalle en las secciones de cada objetivo, a las que remite el texto.

3.2.1 La plataforma DonkeyCar y su simulador

El anteproyecto planteó dos vehículos tipo DonkeyCar con una Raspberry Pi 4 como computador, simulación en CoppeliaSim y visión con OpenCV. DonkeyCar es una plataforma abierta de vehículos a escala con dirección tipo automóvil, orientada a la conducción autónoma con una cámara [58]. La misma lista de herramientas del anteproyecto nombraba ya el AWS DeepRacer como hardware, y la ingeniería inversa sobre esos vehículos como técnica.

Lo primero que se hizo, en abril, fue instalar el simulador de la plataforma y conducir con él. Es una aplicación con pistas exteriores, y el vehículo se maneja desde la interfaz web del propio DonkeyCar, que muestra la imagen de su cámara y permite grabar los recorridos (figura 3.1, izquierda). En paralelo se empezó un modelo propio del vehículo para ROS 2, con el chasis, las cuatro ruedas, la dirección y su árbol de transformaciones, que llegó a representarse en el visualizador de ROS 2 (figura 3.1, derecha).

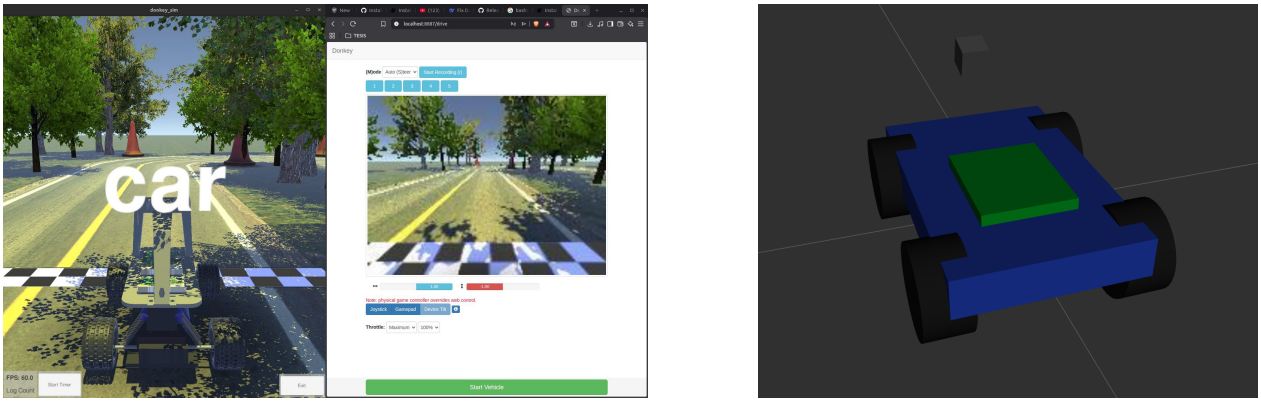


Figura 3.1. Exploración de la plataforma DonkeyCar. A la izquierda, su simulador conducido desde la interfaz web de la plataforma, con la imagen de la cámara del vehículo. A la derecha, el modelo propio del vehículo en el visualizador de ROS 2, con el chasis, las ruedas y la dirección.

Ese trabajo encontró tres limitaciones. El ecosistema DonkeyCar no se actualiza desde hace años y no es compatible con las versiones recientes de ROS 2. Su simulador no se comunica con ROS 2, de modo que no permite probar el mapeo, la localización, la planificación ni la comunicación entre robots, que son el núcleo del proyecto. Y no existía un modelo del DonkeyCar para Gazebo con dirección Ackermann que estuviera mantenido y funcionara en ROS 2 Humble.

Para elegir el simulador se fijaron cuatro criterios: compatibilidad con ROS 2 Humble, vehículos con dirección Ackermann, entornos interiores y soporte para mapeo y navegación. Se consideraron CoppeliaSim, que proponía el anteproyecto, el simulador del curso F1TENTH [59] y Gazebo [52]. Quedó Gazebo, por su integración directa con ROS 2 y con su sistema de control de articulaciones.

La decisión sobre el vehículo se tomó en dos pasos. Primero se resolvió conservar un chasis con dirección tipo automóvil. Como cada robot atiende un piso y la persona cambia de nivel por la escalera, el vehículo no tiene que entrar a un ascensor ni subir escalones. La vuelta dentro del pasillo se podía dejar a un planificador para automóviles, capaz de maniobrar en varios tiempos. Una semana después, por la falta de un modelo de simulación, se cambió al modelo del AWS DeepRacer (sección 3.4.1).

3.2.2 El modelo de AWS DeepRacer y su migración a ROS 2 Humble

AWS publicó en código abierto la descripción del DeepRacer para simulación, con su complemento de cinemática Ackermann para Gazebo, los archivos de configuración y los nodos que traducen las órdenes de velocidad a los servos del vehículo [60]. Ese software estaba escrito para ROS 2 Foxy sobre Ubuntu 20.04, y el proyecto trabaja con Humble sobre Ubuntu 22.04.

La migración se hizo paquete por paquete. Hicieron falta tres cambios, todos por la evolución del sistema de control de articulaciones de ROS 2 entre las dos versiones. La orden que activa los controladores pasó de `start` a `active` en dos archivos de lanzamiento, y el componente que publica el estado de las articulaciones cambió de nombre y de tipo. Los seis paquetes compilaron sin errores. Más tarde se eliminaron los dos avisos de compilación del complemento Ackermann: una cabecera obsoleta y una opción del compilador que la nueva versión no admite.

La validación se hizo en un mundo vacío (figura 3.2, izquierda). El vehículo apareció con el láser y la cámara publicando, a 62 cuadros por segundo y con un factor de tiempo real de 1,00. Los siete controladores quedaron activos: el publicador de estado, las cuatro ruedas y las dos articulaciones de dirección. El vehículo respondió a las órdenes de velocidad, y la odometría registró 0,40 m/s ante una orden de 0,5 m/s, coherente con la respuesta del controlador y la fricción del modelo.

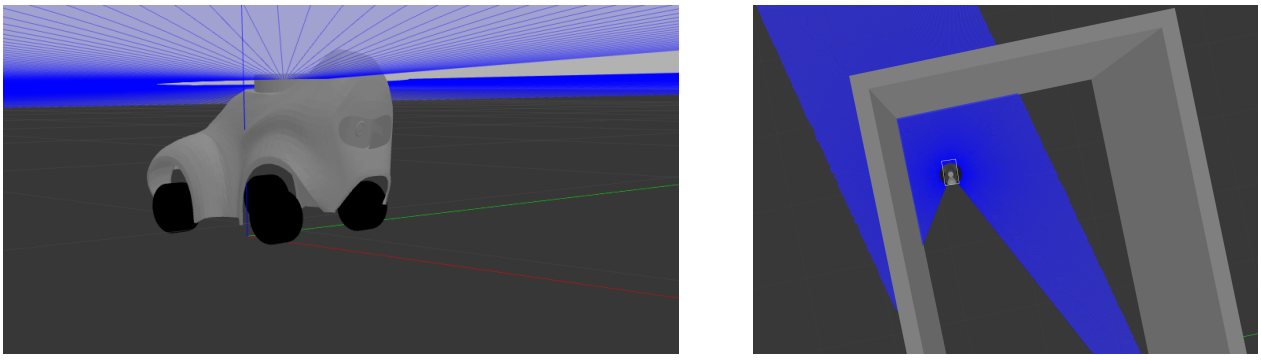


Figura 3.2. El modelo del DeepRacer migrado a ROS 2 Humble. A la izquierda, en un mundo vacío, con el barrido del láser en azul. A la derecha, dentro del primer modelo del pasillo de la universidad, de 7 por 3 m; el sector sin barrido detrás del vehículo es el que no cubría el láser del modelo simulado.

En ese momento el plan era conservar el chasis del DeepRacer y reemplazar su tarjeta, con procesador Intel, por una Raspberry Pi 4 con Humble. Ese plan cambió al empezar el trabajo con los vehículos (sección 3.2.5).

La configuración de Nav2 que se preparó durante el primer semestre también estaba escrita para Foxy, y se migró al empezar el segundo. Cambiaron de nombre el servidor de las maniobras de recuperación, el planificador híbrido, el modelo de movimiento de la localización y los parámetros de los árboles de comportamiento por omisión. La adaptación de Nav2 a un vehículo que no gira sobre su eje está en la sección 3.4.3.

3.2.3 Los primeros entornos simulados y el seguidor de paredes

En mayo se modeló un tramo del pasillo de la universidad con el editor de edificios de Gazebo, de 7 por 3 m, y se condujo dentro el vehículo de forma remota (figura 3.2, derecha). Sobre ese pasillo se programó un seguidor de paredes, que la dirección sugirió como ejercicio de integración entre el láser y la cinemática del vehículo. La primera versión aplicaba control proporcional-derivativo sobre un solo rayo lateral del láser, y saturaba el giro porque no tenía información de la orientación del vehículo respecto a la pared. La segunda aplicó la técnica de dos rayos del curso F1TENTH, que estima además ese ángulo y la distancia a la pared que tendrá el vehículo un metro más adelante.

En el pasillo de 7 por 3 m ninguna sintonía estabilizó al vehículo. El giro se saturaba, y el vehículo trazaba círculos o chocaba con las paredes, porque su radio de giro deja muy poco margen en 3 m de ancho. En un pasillo ampliado de 15 por 5 m el seguidor convergió por tramos, a unos 0,76 m de la pared frente al objetivo de 0,5 m, y se perdía en los extremos. De ahí se concluyó que un comportamiento reactivo no anticipa las restricciones de un vehículo tipo automóvil en un pasillo estrecho, y el proyecto pasó a la navegación con un mapa y un planificador que conoce esas restricciones.

Después se modeló la primera planta del edificio de pruebas, un pasillo de unos 44 por 5 m con las puertas de los salones abiertas. En ella se comprobaron, con el vehículo conducido a distancia, el láser, la cámara, la odometría y el árbol de transformaciones. La cámara quedó disponible y el sistema final no la usa.

Con ese mundo se construyó el primer mapa por localización y mapeo simultáneos (SLAM). Se aceptó entonces sin medirlo. Medido en septiembre, resultó de 23,04 por 15,72 m, la mitad del largo y el triple del ancho del pasillo, deformado por la deriva de la pose estimada. Fue la primera manifestación, sin diagnosticar entonces, de la falta de información de avance en un pasillo recto (sección 2.3.5), que acabó determinando cómo se hicieron los mapas del proyecto (sección 3.4.6).

3.2.4 La integración en simulación

El segundo semestre empezó con un cronograma replanificado, aprobado por el director, con un criterio de cierre para cada semana. Lo primero fue ordenar la base. El espacio de trabajo donde se compilaba el código resultó ser una copia del repositorio del fabricante y no un enlace al del proyecto, de modo que los ajustes del mapeo del primer semestre nunca se habían ejecutado. Se enlazó al repositorio del proyecto y se fijó el contrato de interfaces (sección 3.3.2). La tabla 3.2 resume los pasos siguientes.

Tabla 3.2. Pasos de la integración en simulación, en el orden en que se dieron.

Cuándo	Qué se logró
Inicio de agosto	Nav2 migrado a Humble y adaptado a la dirección Ackermann; contrato de interfaces
Mediados de agosto	Dos agentes aislados, cada uno con su espacio de nombres, en un simulador por vehículo; entorno de dos niveles; mapa de navegación calculado a partir del modelo del edificio
Segunda mitad de agosto	Orden de velocidad corregida en el modelo del vehículo; mundo definitivo de dos pisos, después partido en un mundo por piso; requisitos numerados y protocolo experimental
Finales de agosto	Los dos agentes navegando a la vez; coordinador y planificador; primera misión completa y primera misión con relevo entre los dos pisos
Septiembre	Campaña de treinta misiones; interfaz web operada desde un teléfono; confirmación de cambio de piso y cancelación; congelación del código

Dos de esos pasos cambiaron el diseño. El primero fue la separación de los vehículos en dos simuladores. En un mismo proceso de Gazebo no fue posible tener dos robots, porque la integración de Gazebo con ROS 2 en Humble aplica a todos los complementos el espacio de nombres del primer modelo cargado. Por eso cada vehículo tiene su propio simulador, y más tarde cada piso, su propio mundo (sección 3.3.5).

El segundo fue el mapa de navegación. El mapa que se había construido con SLAM cubría el 35,5 % del largo del pasillo, medía 15,0 m de ancho frente a 5,3 m reales, y 2688 de sus 3618 celdas ocupadas, el 74 %, no tenían ninguna pared detrás. Se reemplazó por un mapa calculado a partir de la geometría del modelo del edificio, que es exacto en simulación. Como un mapa de ocupación es un corte horizontal, el del piso superior resultó idéntico y no hubo que levantarlo.

En la misma etapa se corrigió un defecto del modelo del vehículo. La orden de velocidad estándar de ROS 2 expresa el giro como velocidad angular, y el complemento de simulación la tomaba como el ángulo del volante. Con la corrección, el ángulo de dirección se calcula con el modelo de bicicleta a partir de la velocidad angular, la velocidad lineal y la distancia entre ejes, y el error de seguimiento de las trayectorias bajó a la mitad. La misma corrección se aplicó al nodo que mueve el vehículo real.

3.2.5 Del vehículo físico a la campaña con los dos vehículos

El trabajo con los vehículos físicos corrió en paralelo desde agosto. Al empezar el semestre se instaló el software abierto de la comunidad del DeepRacer [61] en una Raspberry Pi 4 y en la tarjeta original del vehículo, y se probó la marcha desde la interfaz web del vehículo. Los vehículos quedaron operando con su tarjeta original, con Ubuntu 24.04 y ROS 2 Jazzy (sección 3.4.2). Como la simulación no puede pasar a Jazzy, el proyecto trabaja desde entonces con dos distribuciones de ROS 2 (sección 2.3.2). La tabla 3.3 resume el resto del camino.

Tabla 3.3. Pasos del trabajo con los vehículos físicos, en el orden en que se dieron.

Cuándo	Qué se logró
Agosto	Software de la comunidad en los vehículos y marcha desde su interfaz web; diagnóstico del láser y del mando desde ROS 2; láser funcionando junto al software del fabricante; primer mapa del laboratorio
Mediados de septiembre	Llegada del segundo vehículo y comunicación entre los dos; banda muerta de tracción medida
Finales de septiembre	Decisión de intentar la demostración física, con seis pruebas de verificación; odometría con el láser; mapa construido a bordo y primera navegación autónoma de un vehículo; los dos vehículos aislados entre sí; primeras pruebas en el piso 2
Inicio de octubre	Paso a los pisos 3 y 4, levantados con flexómetro; odometría verificada; unidad inercial integrada; red de un punto de acceso por piso
Octubre	Llegada verificada en el piso 4; media vuelta en varios tiempos; recorrido del piso 3; coordinador en el computador portátil; misión con relevo y campaña con los dos vehículos

Tres hechos de ese camino cambiaron el plan. Durante tres semanas el segundo vehículo pareció averiado, y la causa eran los permisos con que corría el software del fabricante (sección 3.4.9). En

el hall del piso 2 se comprobó con flexómetro la primera medida del modelo, y el modelo daba el hall un 25 % más ancho que el real. Como las demás medidas nunca se habían comprobado, las pruebas pasaron a los pisos 3 y 4, levantados con flexómetro (sección 3.4.6). Y con las dos cadenas de navegación encendidas, la tarjeta de un vehículo se saturó y Nav2 no arrancaba, de modo que el coordinador y el grabador pasaron al computador portátil (sección 3.4.9).

3.3 Arquitectura funcional del sistema

Esta sección corresponde al primer objetivo específico: modelar la arquitectura funcional, con sus requisitos, sus escenarios de operación en varios pisos, la estrategia de asignación de tareas y el esquema de comunicación entre robots.

La figura 3.3 muestra el sistema tal como se implementó en simulación, donde se desarrolló y se evaluó primero. Están la interfaz por la que entra la solicitud, el coordinador que la reparte, los dos agentes con su pila de navegación y la escalera donde ocurre el relevo.



Figura 3.3. Arquitectura del sistema en simulación, con un proceso de Gazebo por piso. Ningún robot atraviesa la zona de transición: lo que la cruza es el usuario, y lo que se traspasa entre agentes es la responsabilidad de guiarlo.

Al llevar el sistema a los dos vehículos, la estructura se mantuvo y cambió la implementación de varios bloques. La sección 3.3.6 reúne esos cambios y sus causas.

3.3.1 Requisitos

El anteproyecto no enumeró los requisitos del sistema, aunque su metodología pedía verificar la implementación contra ellos. Se reconstruyeron al iniciar la integración a partir de tres fuentes: los objetivos específicos, el alcance y el contrato de interfaces. Cada requisito cita su fuente y tiene una prueba que lo declara cumplido o no, de modo que un requisito sin prueba no entra en la matriz.

La matriz tiene treinta y seis requisitos. Veintinueve describen funciones del sistema, y siete

son restricciones del alcance, como operar en interiores, en condiciones controladas, o correr la simulación en tiempo real. El primer objetivo tiene diez requisitos, sobre arquitectura, coordinación y relevo. El segundo tiene seis: locomoción, sensado, odometría, mando desde ROS 2, comunicación entre los vehículos y código común. El tercero tiene seis, sobre la interfaz, la confirmación de cambio de piso y la cancelación. Y el cuarto tiene siete: las cuatro métricas, el registro de cada misión y las dos campañas de evaluación. La confirmación de cambio de piso y la cancelación no estaban en el anteproyecto: las pidió el director al ver el sistema en uso. El avance de cada objetivo se calcula como el promedio del estado de sus requisitos.

3.3.2 El contrato de interfaces

Antes de escribir código se fijó un contrato: qué habla con qué y en qué formato. El sistema tiene tres piezas. Un coordinador central decide y manda. Dos agentes, uno por nivel, ejecutan. Y una interfaz web es la vía por la que el usuario pide el servicio. La comunicación sigue tres reglas:

1. El coordinador manda a un agente solo mediante la acción de navegación de ROS 2; no publica velocidades directamente. En la versión final puede pedirle además una media vuelta, que el agente ejecuta por su cuenta antes de navegar (sección 3.4.10).
2. La interfaz habla solo con el coordinador, nunca con los agentes.
3. El texto que ve el usuario lo redacta el coordinador, y la interfaz lo muestra literal.

La tercera regla evita que la lógica de presentación se duplique en el navegador. Si el cliente interpretara el estado numérico para decidir qué mostrar, habría dos copias de la misma lógica y acabarían divergiendo.

El contrato se implementó como un paquete propio de mensajes de ROS 2. La acción de guiado recibe el origen y el destino que eligió el usuario. Durante la misión devuelve como retroalimentación el estado completo, y al final, si tuvo éxito, el tiempo total y el número de relevos o, si falló, el motivo. El mensaje de estado de la misión lleva la etapa, el agente activo, el punto al que va en ese momento, la distancia restante y el texto para el usuario. Dos mensajes más describen los puntos de interés del catálogo y el estado de cada agente.

La misión avanza por las etapas de la tabla 3.4. Cada cambio de etapa se publica, y esas publicaciones son las marcas de tiempo con que se calculan las métricas de la evaluación.

Tabla 3.4. Etapas de una misión, tal como las publica el coordinador.

Etapa	Significado
Recibida	El coordinador aceptó la solicitud y todavía no asignó agente
Primer tramo	El agente del nivel de origen va a recoger al usuario y lo guía
Transferencia	El agente del otro nivel va a su punto de transferencia
Esperando confirmación	El sistema espera que el usuario confirme que ya cambió de piso
Segundo tramo	El agente del nivel de destino guía al usuario hasta el final
Completada	La misión terminó con la llegada verificada
Fallida	La misión terminó sin cumplirse, con el motivo en el texto
Cancelando	El usuario canceló y el agente activo regresa a su punto de transferencia
Inactiva	No hay misión en curso

3.3.3 El planificador, escrito sin ROS

El relevo es el aporte declarado del proyecto. Por eso se escribió como función pura: entran el catálogo de puntos y un par origen-destino, y salen los tramos que hay que ejecutar. No depende de la capa de comunicaciones de ROS 2 ni mantiene estado.

Hay dos formas de plan. Si el origen y el destino están en el mismo nivel, la misión tiene dos tramos del mismo agente: ir a recoger al usuario y llevarlo al destino. Si están en niveles distintos, tiene cuatro: los dos anteriores, con el segundo terminando en el punto de transferencia del nivel de origen; el desplazamiento del segundo agente a su punto de transferencia; y el guiado final. El punto de encuentro no se negocia: es el punto de transferencia declarado de cada nivel en el catálogo.

La razón para escribirlo así es de verificabilidad. Si el relevo viviera dentro del nodo, mezclado con clientes de acción, la única forma de comprobarlo sería levantar dos simuladores y observar si el vehículo llega, lo que es lento y frágil. Como función pura, la prueba recorre las 930 combinaciones de origen y destino del catálogo y verifica cada plan en milisegundos, sin simulador.

Esa prueba encontró defectos que una corrida no habría delatado. El más claro: cuando el origen o el destino es el propio punto de transferencia de su nivel, el plan mandaba al mismo robot dos veces a la misma pose. Un desplazamiento de cero metros no falla en Nav2, que devuelve éxito al instante, así que la simulación no lo habría mostrado.

3.3.4 La asignación de tareas

Cada nivel tiene un agente asignado, y esa correspondencia es un dato de configuración, no una constante del código. El coordinador elige el agente al aceptar la solicitud, según el nivel del origen y del destino. La sección 2.3.1 clasifica esa asignación y muestra por qué no hace falta un optimizador: con un solo agente admisible por nivel, el método húngaro devuelve la misma tabla.

El planificador exige un único punto de transferencia por nivel, y falla con un mensaje explícito si encuentra dos. Hoy cada planta tiene una escalera y eso se cumple. La verificación está para que, el día que haya dos, la decisión se tome a la vista.

En los vehículos, el coordinador comprueba cada llegada por su cuenta, con la pose que publica el agente, en lugar de confiar en el resultado de Nav2. Nav2 declara la llegada en cuanto el vehículo entra en su margen, y el vehículo sigue rodando un poco más. En las llegadas medidas en el piso 4, el vehículo estaba a entre 0,53 y 0,62 m de la meta en la última orden de avance, y un segundo después, ya quieto, a entre 0,09 y 0,53 m. Por eso el coordinador espera 1,5 s antes de medir.

3.3.5 Escenarios de operación en simulación

Los escenarios con varios pisos se construyeron primero en simulación, en Gazebo 11 con ROS 2 Humble, con el modelo del AWS DeepRacer migrado a esa versión (sección 3.2.2). El entorno evolucionó en varios pasos. Tras los pasillos de prueba y la primera planta del edificio (sección 3.2.3), esa planta se duplicó para obtener dos niveles. Más tarde se adoptó un modelo más fiel del pasillo, cuyo catálogo llegó a 31 puntos de interés: quince en el piso 1 y dieciséis en el piso 2. Por último, ese mundo se partió en uno por piso.

La partición respondió a una interferencia medida. Con los dos pisos en un mismo mundo, el láser simulado, de 10 m de alcance, veía paredes del otro piso en 61 de 1701 posiciones muestreadas. Esas posiciones estaban en el tramo donde arranca el primer vehículo y donde está su punto de transferencia. Desde entonces cada piso es un mundo distinto, con su propio proceso de simulación, y los dos vehículos comparten un dominio de comunicaciones, separados por espacio de nombres (sección 2.3.6). La figura 3.4 muestra una misión con relevo entre los dos procesos.

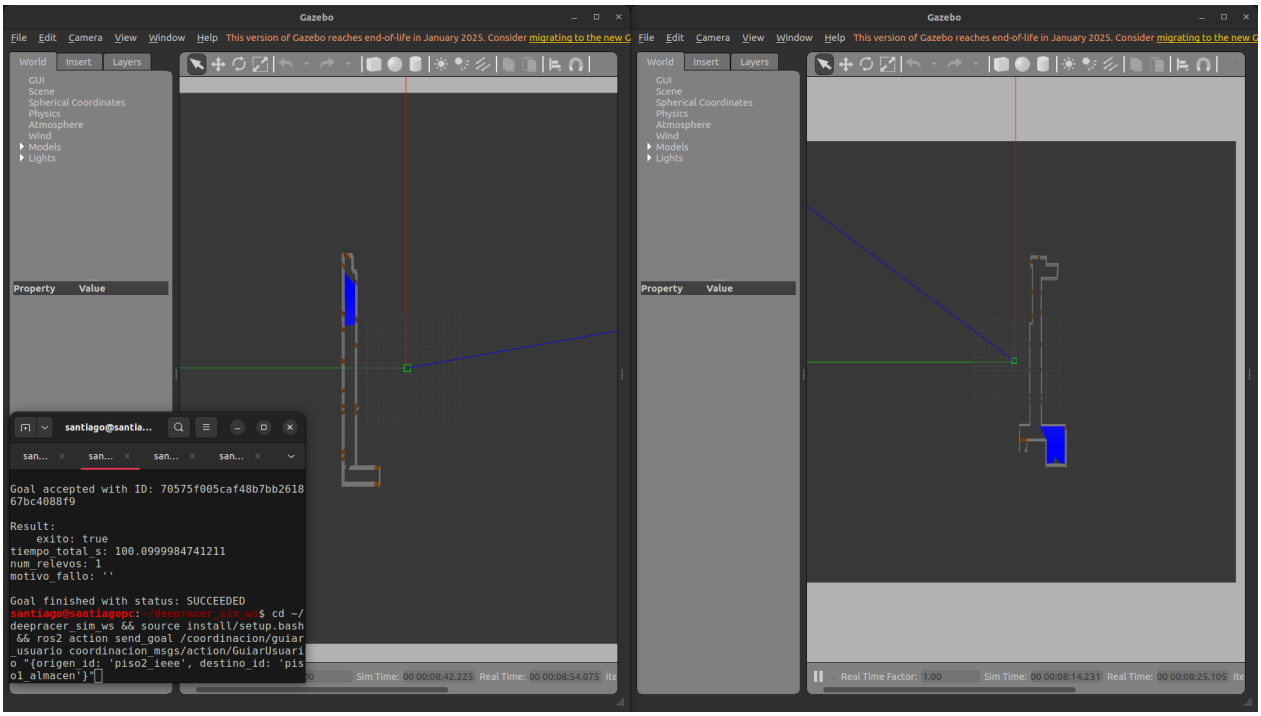


Figura 3.4. Misión con relevo en simulación. Cada ventana es un proceso de Gazebo con un piso y su vehículo; el láser de cada uno se ve en azul. La terminal muestra el resultado de la acción: éxito, un relevo y 100,1 s de duración.

La simulación debe correr a tiempo real para que las métricas temporales valgan. En las treinta misiones de la campaña, el factor de tiempo real tuvo un mínimo de 0,9917 y una media de 0,9973.

3.3.6 De la simulación a los vehículos

Al llevar el sistema a los vehículos se mantuvo su estructura: un coordinador, un agente por piso, el relevo en la escalera y las reglas de comunicación del contrato (sección 3.3.2). También se mantuvo el código. El del coordinador, el del planificador, el de los mensajes y el de la página es el mismo en los dos destinos: compila sin cambios en las dos distribuciones de ROS 2 y completó una misión en cada uno. La configuración de Nav2 tiene una variante por distribución, derivada de una base común, y una prueba comprueba que las dos no se separen.

Lo que cambió fue la implementación de los bloques. La tabla 3.5 reúne cada cambio con su causa y la sección donde se trata, y la figura 3.5 muestra el sistema como funcionó en la campaña con los dos vehículos.

Tabla 3.5. Cambios en la implementación al pasar de la simulación a los vehículos.

Parte	En simulación	En los vehículos	Causa
Distribución de ROS 2	Humble en todos los componentes	Jazzy en los vehículos y en el coordinador, que corre en un contenedor del portátil; Humble en la página y en rosbridge	El software de la tarjeta original solo existe para Ubuntu 24.04 con Jazzy, y la acción de navegación de Jazzy no es compatible con la de Humble (sección 2.3.2)
Coordinador	En el portátil, junto a los dos simuladores	En el portátil, después de un intento en uno de los vehículos	Con el coordinador y el grabador en una tarjeta y las dos navegaciones encendidas, la carga subió de 6 a entre 17 y 33 y Nav2 no arrancaba (sección 3.4.9)
Comunicación	Un solo computador y un dominio de comunicaciones; un espacio de nombres por agente	Red inalámbrica con un punto de acceso de 5 GHz por piso; una partición por vehículo y topología en estrella hacia el portátil	Los tópicos del fabricante no llevan espacio de nombres, y unos cuarenta procesos anunciándose por la red saturaban las tarjetas (sección 3.4.9)
Locomoción	Complemento Ackermann de Gazebo con `ros2_control`	Puente de la orden de velocidad a los servos, con una escala por vehículo	La tracción real tiene banda muerta y funciona en lazo abierto (sección 3.4.7)
Sensado	Láser simulado de 10 m de alcance	RPLIDAR A1M8 y la unidad inercial de la tarjeta; cámaras apagadas	Las cámaras no se usan y cargaban la tarjeta (sección 3.4.2)
Odometría	La del simulador	La del láser, combinada con la unidad inercial mediante un filtro	El vehículo no tiene codificadores en las ruedas, y con el láser solo el rumbo derivaba 5,9° por minuto (secciones 3.4.5 y 3.4.8)
Mapa	Calculado de la geometría del mundo simulado	Derivado de la planta medida con flexómetro	El mapeo con SLAM a bordo no dio un mapa utilizable en ningún pasillo (sección 3.4.6)
Escenario	Pisos 1 y 2 del modelo del edificio	Pisos 3 y 4, levantados con flexómetro	El modelo del piso 2 no correspondía al edificio, y en sus pasillos uniformes la odometría del láser no veía el avance (sección 3.4.6)
Llegada	Tolerancia de 0,25 m	Tolerancia de 0,5 m, verificada con la pose del agente 1,5 s después de que Nav2 da la meta por alcanzada	Por la banda muerta, el vehículo no alcanza la precisión de la simulación, y sigue rodando cuando Nav2 declara la llegada (secciones 3.4.7 y 3.3.4)
Media vuelta	La planifica Nav2, con marcha atrás	Maniobra propia del agente, vigilada con el láser y el mapa	En la salida del piso 4, de 2,5 m de ancho, la media vuelta de Nav2 abortó contra la pared (sección 3.4.10)

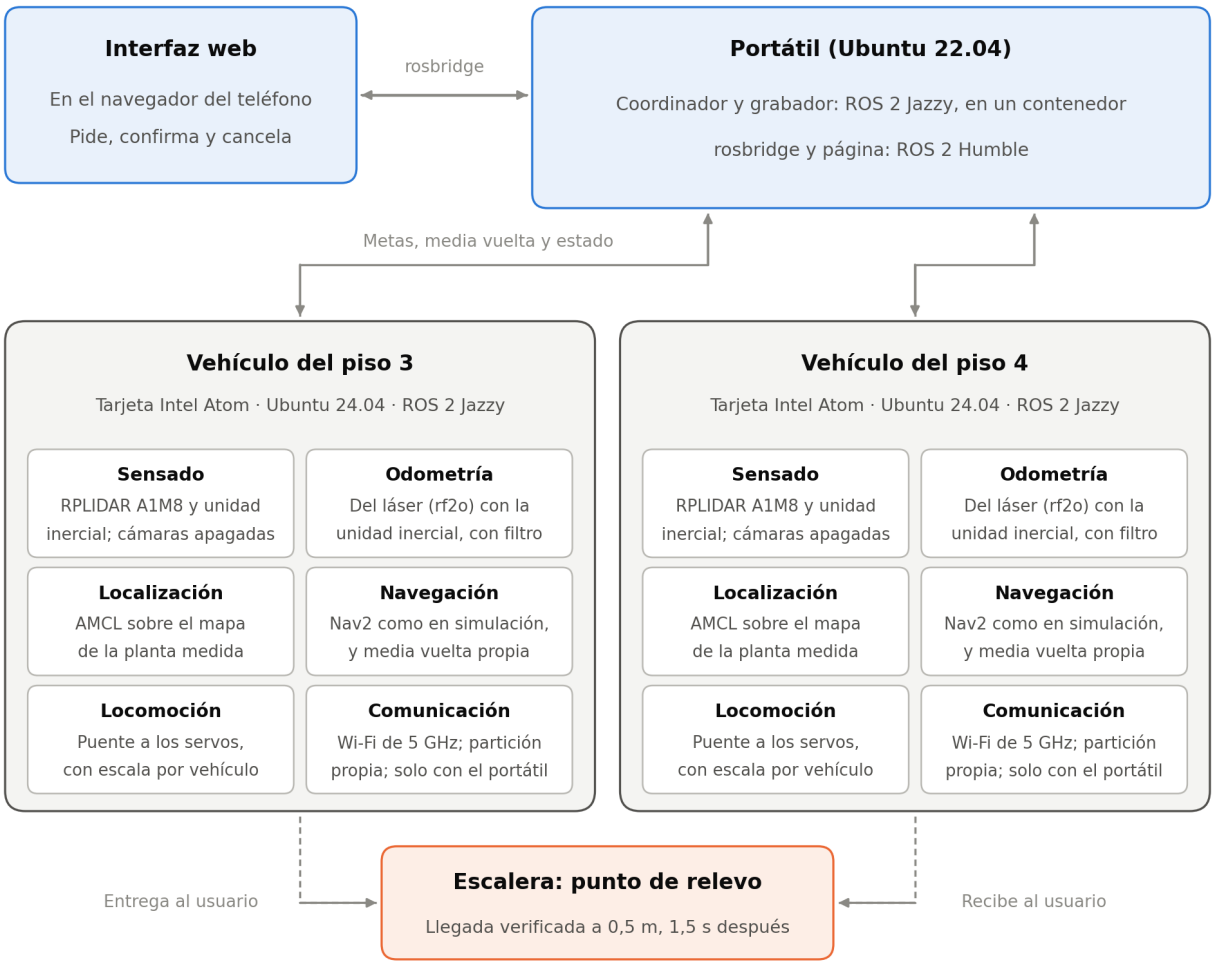


Figura 3.5. Arquitectura del sistema en la campaña con los dos vehículos, en los pisos 3 y 4. La estructura es la de la figura 3.3; cambia la implementación de los bloques.

3.4 Plataforma robótica

Esta sección corresponde al segundo objetivo específico: desarrollar una plataforma robótica de dos vehículos, con los módulos de locomoción, sensado, procesamiento y comunicación que necesita la operación colaborativa.

3.4.1 Selección de la plataforma: de DonkeyCar a DeepRacer

El objetivo habla de «dos vehículos tipo DonkeyCar». Los vehículos empleados son dos AWS DeepRacer, y el cambio se decidió durante el diseño conceptual, por las tres limitaciones que

encontró el análisis de la plataforma DonkeyCar (sección 3.2.1).

El DeepRacer resolvía las tres. Es la misma clase de plataforma: un chasis de radiocontrol a escala 1:18 con dirección Ackermann, un computador a bordo y software abierto. AWS publicó en código abierto su modelo para simulación, con la descripción del vehículo y un complemento de cinemática Ackermann para Gazebo. Y la universidad ya tenía dos unidades. Lo que el objetivo fija, que es cinemática tipo automóvil, bajo costo y código modificable, se cumple igual con una plataforma que con la otra. Usar las que existían dejó el tiempo para el problema que el proyecto declara como suyo, que es la coordinación.

El cambio tuvo un costo. El DeepRacer trae su propia pila de software, pensada para conducción autónoma por visión y para competencia. Buena parte del trabajo con los vehículos consistió en convivir con ella, porque sus nodos ocupan el láser, se ejecutan con el usuario administrador y publican sin espacio de nombres (sección 3.4.9).

3.4.2 Componentes del vehículo

Durante el diseño se planeó reemplazar la tarjeta de cómputo de los vehículos por una Raspberry Pi 4. Al empezar la integración se instaló en las dos tarjetas el software de la comunidad que adapta el DeepRacer a ROS 2, y los dos vehículos operan desde entonces con su tarjeta original. Para esa tarjeta el software solo se publica sobre Ubuntu 24.04 con ROS 2 Jazzy, y de ahí que el sistema use dos distribuciones (sección 2.3.2). La tabla 3.6 reúne los componentes.

Tabla 3.6. Componentes de cada vehículo y del computador central.

Componente	Descripción
Chasis	AWS DeepRacer, escala 1:18, dirección Ackermann, motor de tracción con variador y servomotor de dirección
Cómputo	Tarjeta original, procesador Intel Atom E3930 de dos núcleos, 4 GB de memoria; Ubuntu 24.04 y ROS 2 Jazzy
Láser	RPLIDAR A1M8-R5: 360°, 360 muestras por barrido, alcance de 0,15 a 12 m, entre 6,8 y 7 Hz
Unidad inercial	Bosch BMI160 integrada en la tarjeta, por I2C: giroscopio y acelerómetro
Cámaras	Estéreo; desactivadas por software, porque el sistema no las usa y cargaban la tarjeta
Alimentación	Una batería para el cómputo y otra de polímero de litio para la tracción
Computador central	Portátil con Ubuntu 22.04 y ROS 2 Humble (AMD Ryzen 5 PRO 5650U, 16 GB); el coordinador corre en un contenedor con Jazzy

Cada vehículo atiende un piso: uno el 3 y otro el 4. El láser que traía el kit no era el que describía el primer informe de hardware; la identificación correcta se hizo leyendo el modelo que reporta el propio sensor.

3.4.3 Nav2 sobre un vehículo que no gira sobre su eje

La plataforma sigue cinemática tipo automóvil y no puede girar en el sitio. La configuración por omisión de Nav2 asume tracción diferencial, de modo que propone trayectorias que el vehículo no puede ejecutar. Por eso se sustituyeron dos de sus tres capas, con los componentes que justifica la sección 2.3.3. La planificación global pasó al planificador híbrido A* de Smac con modelo Reeds-Shepp, que admite marcha atrás y genera las maniobras de inversión que el vehículo necesita. El seguimiento pasó al controlador de persecución pura regulada, con marcha atrás habilitada y velocidad deseada de 0,5 m/s. Del árbol de comportamiento se quitó la recuperación que gira sobre el propio eje, porque el vehículo no puede hacerla: sin ese cambio, ante un bloqueo, el vehículo se quedaba moviendo las ruedas sin avanzar hasta agotar los reintentos.

Los parámetros de Nav2 se mantienen en dos variantes, una por distribución, derivadas de una base común. En Jazzy cambió la forma de nombrar los complementos y algunos parámetros. Una prueba automática compara las dos variantes y falla si se separan en algo que no sea una de esas diferencias

documentadas.

3.4.4 El orden de puesta en marcha

Cada pieza de la cadena de navegación solo se puede diagnosticar si la anterior funciona. La guía oficial de puesta en marcha de Nav2 [62] lo plantea como una escalera de siete pasos. De abajo arriba: transformaciones entre marcos de referencia, odometría, barrido del láser, mapa, localización, mapas de costos y, al final, planificador y controlador.

El proyecto la siguió en ese orden, y eso permitió aislar el problema del apartado siguiente. Sin la escalera, una odometría que no mide se confunde con un planificador que no planifica, porque los dos se manifiestan igual: el vehículo no llega. Con ella, el problema queda acotado a un paso.

El orden importa además al arrancar. Los mapas de costos leen el mapa al configurarse, y si el servidor de mapas no está activo en ese instante, la capa estática queda vacía. Entonces el planificador rechaza cualquier meta indicando que el punto de partida está ocupado: el mensaje señala el origen, y la causa está tres pasos más abajo. El programa de arranque de los vehículos activa el servidor de mapas antes que los servidores de Nav2, y espera a que los seis nodos de la cadena estén activos antes de declararla lista. En las pruebas de laboratorio con los dos vehículos, la cadena completa quedó lista en 4 min 4 s en uno y en 4 min 22 s en el otro.

3.4.5 La odometría del láser como limitación principal

El vehículo no tiene codificadores en las ruedas. Su única fuente de odometría es la comparación entre barridos sucesivos del láser, y durante meses esa pieza no dio una medida utilizable.

La causa estaba en la información disponible (sección 2.3.5). En un pasillo recto y de sección uniforme, cada barrido es prácticamente idéntico al anterior a lo largo del eje del corredor, y el estimador devuelve un desplazamiento cercano a cero. Medida sobre el edificio, en los pasillos de los pisos 1 y 2 la odometría registraba entre el 1,3 % y el 73 % del avance real, según el tramo.

La figura 3.6 muestra lo que eso produce. Las tres trayectorias son del mismo vehículo en el mismo hall. La localización se desplaza hasta más de dos metros respecto de donde el vehículo estaba, medido con flexómetro, y en una de las corridas sale del recinto. La línea a trazos marca otro resultado de esa misma sesión: la pared norte estaba modelada 0,84 m más al norte de lo que está.

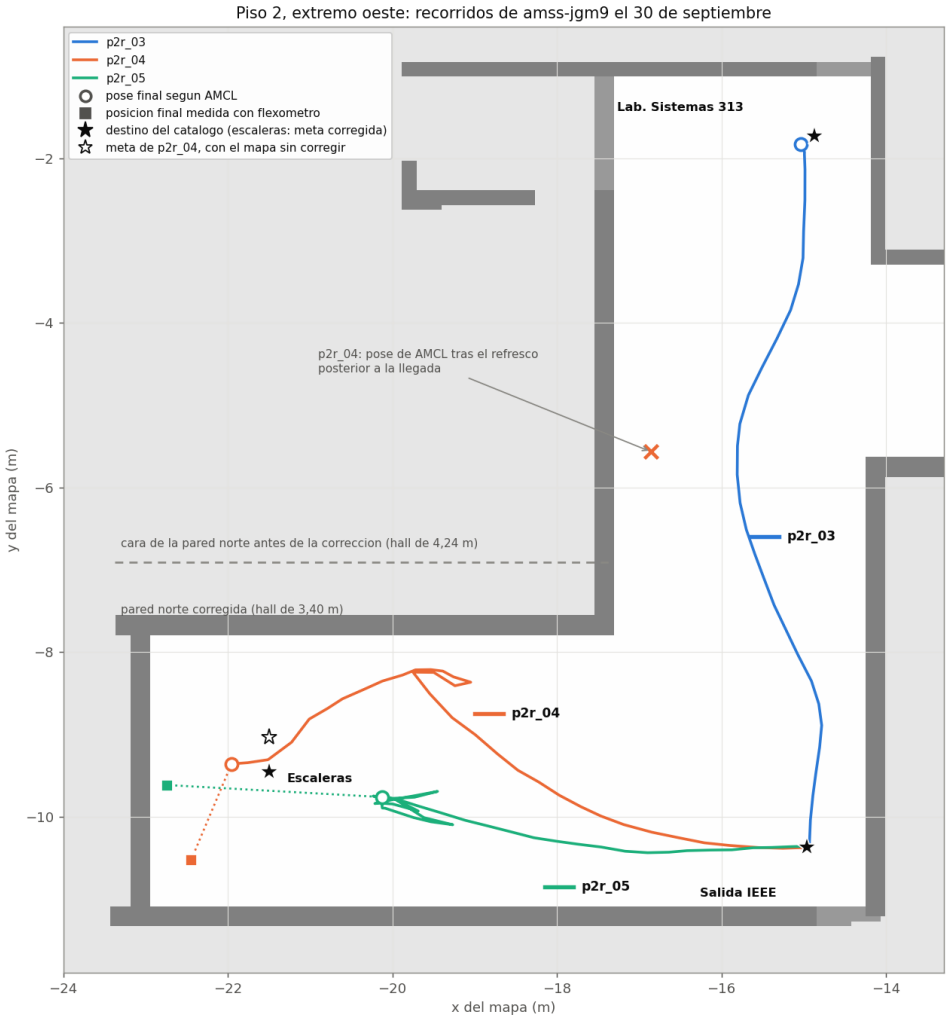


Figura 3.6. Tres recorridos en el hall del piso 2. El círculo es la pose que reporta la localización al terminar; el cuadrado, la posición real medida con flexómetro. La separación entre los dos es el error, y su causa está en la odometría, no en el algoritmo de localización.

La solución fue cambiar de entorno, porque ningún ajuste del algoritmo recupera información que la señal no contiene. El criterio para elegirlo salió de la misma medida: hace falta estructura no uniforme dentro del alcance del sensor durante todo el recorrido. Los pisos 3 y 4 la tienen. Entre las tres hornacinas de los salones, los casilleros, la papelera, el ascensor y la escalera suman ocho cambios de perfil en 25 m, uno cada tres metros. Con la misma odometría y sin cambiar un parámetro, el error cayó a menos del 5 %: $+3{,}3\,\%$ sobre 14,57 m y $-4{,}6\,\%$ sobre 6,68 m, medidos con flexómetro en la primera sesión en esos pisos. Los directores aceptaron el cambio de sitio.

3.4.6 Los mapas, derivados de medidas y no de SLAM

El proyecto intentó durante semanas construir el mapa con SLAM sobre los vehículos y no lo consiguió en ningún pasillo, por la misma razón del apartado anterior. La vía que funcionó fue otra: levantar la planta con flexómetro y derivar de ese levantamiento tanto el mundo simulado como el mapa de navegación.

Eso tiene una consecuencia. Simulación y edificio coinciden porque salen de la misma fuente, no porque se parezcan. El proyecto había dado esa correspondencia por supuesta, y al verificarla por primera vez resultó falsa: el modelo anterior daba un hall 0,84 m más ancho de lo que es, un 20 % de error.

La herramienta que genera las plantas incluye una comprobación obligatoria. Un pasillo mide lo mismo por sus dos lados, así que suma las dos paredes y avisa si no cierran dentro del 2 %. Esa comprobación encontró cinco errores antes de generar nada, y las dos plantas cierran al 0,32 % y al 0,62 %.

La figura 3.7 es el resultado para una de ellas. Es el mapa de ocupación con el que navega el vehículo, generado a partir de las medidas. Los entrantes de los salones, los casilleros, el ascensor y la escalera son los ocho cambios de perfil que hacen observable el avance. Cada piso tiene cinco puntos de interés en el catálogo: tres salones, el ascensor y la escalera, que es su punto de transferencia.

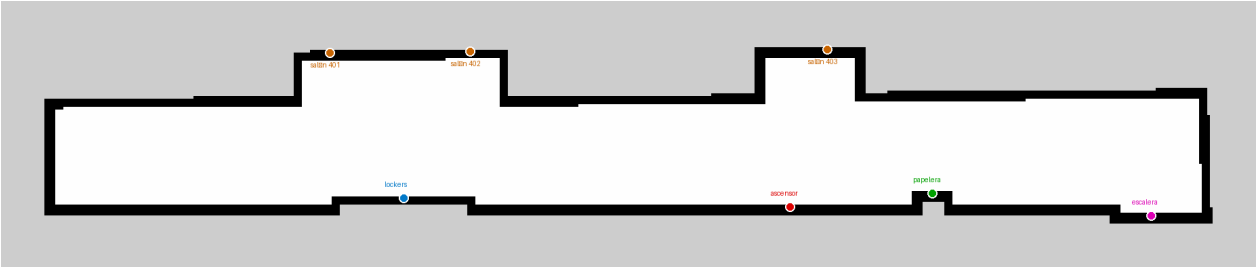


Figura 3.7. Mapa de navegación del piso 4, derivado del levantamiento con flexómetro. El pasillo corre 25 m del extremo norte al sur; los salones quedan al este, y el ascensor y la escalera al oeste.

3.4.7 La banda muerta de tracción y lo que obliga a configurar

Al integrar la cadena de tracción apareció una característica de la plataforma que condiciona la aproximación al destino: el vehículo no se mueve con velocidades pedidas por debajo de 0,40 m/s. El puente que traduce la orden de velocidad a la señal del variador las convertía en tracción nula, y lo hacía sin emitir ningún aviso: desde fuera parecía que la orden se había ejecutado.

Eso forzó una decisión de configuración con consecuencias. La velocidad mínima de aproximación del controlador se subió a ese mismo valor, 0,40 m/s. Sin ese ajuste el vehículo se detenía al entrar en la fase final del recorrido y se quedaba ahí, intentando moverse a una velocidad que su propia cadena anulaba. Después se modificó el puente para que envíe las órdenes inferiores a 0,40 m/s con el escalón de tracción más bajo, de modo que el vehículo ya no se detiene en la fase final.

El efecto es que el vehículo se aproxima a la meta a 0,40 m/s y no puede ir más despacio: no tiene régimen de aproximación fina. Es un límite de la plataforma y no se corrige afinando el controlador. La relación entre la orden y la velocidad real tampoco es la misma en los dos vehículos, porque la tracción funciona en lazo abierto. Por eso el puente tiene una escala por vehículo, ajustada en campo: 0,85 en el del piso 3 y 0,9 en el del piso 4. Con la escala 1,0, el del piso 4 iba a 1,36 m/s y se pasaba de la meta; con 0,9 va a unos 0,7 m/s.

Quedaban dos salidas, y las dos eran decisiones y no ajustes. La tolerancia de llegada en los vehículos reales la fijó el director en 0,5 m, mientras la simulación conserva 0,25 m. Bajar el umbral de arranque del vehículo, por mecánica o por calibración de los servos, queda planteado en los trabajos futuros.

3.4.8 La unidad de medición inercial de la tarjeta

Al principio el vehículo estimaba también su rumbo con la odometría del láser, y en un pasillo recto esa estimación pierde los giros igual que pierde el avance. Con el vehículo quieto, el rumbo de la odometría del láser derivó hasta 5,9° por minuto, y en un giro de 90° hecho a mano registró 19,1°.

La tarjeta de los dos vehículos incluye una unidad de medición inercial, una Bosch BMI160 que el software del fabricante no usa. Se comprobó leyéndola directamente: el giroscopio midió giros de 90° y 360° con un error de 0,1 % a 1,3 %, y quieto no se desvió más de 0,3° en 60 s.

Se escribió un nodo propio que la lee y calibra el sesgo del giroscopio al arrancar, con el vehículo quieto. Sus lecturas se integraron con un filtro de Kalman extendido que combina las dos fuentes según lo que cada una mide bien (sección 2.3.5): de la odometría del láser toma el avance y de la unidad inercial el giro. Para no sobrecargar la tarjeta, la unidad inercial publica a 25 Hz y el filtro corre a 15 Hz. En un giro de 90° contra una línea del piso, el filtro midió 90,17° en un vehículo y 88,38° en el otro, y con el vehículo quieto su rumbo se movió 0,3° o menos por minuto. La posición no mejora con la unidad inercial, porque el filtro la sigue tomando del láser.

Con el rumbo estable se pudo reducir de 1,0 m a 0,5 m el margen en el que Nav2 da la meta por alcanzada, y con ese margen se superó la prueba de navegación de un vehículo (sección 4.2.2).

3.4.9 Los dos vehículos en la misma red

Con los dos vehículos encendidos aparecieron problemas que con uno solo no existían. El primero fue de permisos. El software del fabricante se ejecuta con el usuario administrador, y la capa de comunicaciones de ROS 2 intercambia los datos entre procesos del mismo equipo por memoria compartida, a la que solo ese usuario puede escribir. Los programas lanzados con un usuario normal detectaban los nodos del fabricante, pero no podían intercambiar mensajes con ellos, y ningún error lo advertía. Durante tres semanas eso se interpretó como una avería de uno de los vehículos.

El segundo fue de nombres. Los tópicos del software del fabricante no llevan espacio de nombres, así que una orden de movimiento llegaba a los dos vehículos y la odometría de cada uno recibía el láser del otro. Para resolverlo, cada vehículo tiene una partición de comunicaciones propia, que deja esos mensajes dentro del vehículo, y la cadena del proyecto corre en un espacio de nombres distinto en cada uno.

El tercero apareció en el primer intento de misión con relevo con los dos vehículos. Con las dos cadenas completas, unos cuarenta procesos se anunciaban unos a otros por la red inalámbrica. La tarjeta de un vehículo pasó de una carga de 6 a una de entre 17 y 33, con el procesador al 84–95 %, y Nav2 no llegaba a arrancar. El aumento estaba en los procesos que hablan con la red, mientras la navegación gastaba menos que antes.

La solución fue la topología en estrella del diseño (sección 2.3.2). Los procesos internos de cada vehículo solo se buscan dentro de él. Lo que el coordinador necesita de un vehículo, que es su navegación, su estado y su media vuelta, solo se anuncia al computador portátil, y los vehículos no se ven entre sí. El coordinador y el grabador de las misiones pasaron del vehículo al portátil, dentro de un contenedor con Jazzy, porque la acción de navegación de Jazzy no es compatible con la de Humble. Cada vehículo publica su pose en el mapa dos veces por segundo, y es lo que el coordinador usa para verificar las llegadas.

En las pruebas de laboratorio con todo el sistema, cada tarjeta quedó al 54 % y al 59 % de uso en reposo, y al 68 % y al 74 % navegando, con las ruedas en el aire. El controlador de Nav2, que corre a 10 Hz, se atrasó una vez en un vehículo y cinco en el otro, sin bajar de 5,5 Hz. La red tiene un punto de acceso por piso en la banda de 5 GHz, los dos en la misma subred. Entre los dos vehículos, el tiempo de ida y vuelta de los mensajes de ROS 2 tuvo una mediana de 5,99 ms y un percentil 95 de 11,12 ms, sin pérdidas. El requisito de comunicación entre los vehículos fija un límite de 100 ms para la mediana.

3.4.10 La media vuelta en varios tiempos

Entre misiones el vehículo no vuelve a su punto de partida: cada misión empieza donde terminó la anterior, por acuerdo con los directores. Con frecuencia el siguiente usuario está detrás del vehículo, y un vehículo con dirección Ackermann no puede girar en el sitio. En pasillos de 2,5 a 3,2 m de ancho tiene que dar la vuelta en varios tiempos.

En las primeras pruebas en el piso 4, a la salida del pasillo, de 2,5 m de ancho, la media vuelta de Nav2 abortó dos veces contra la pared: el planificador proponía cada segundo una maniobra distinta. Además, con la misma orden de velocidad el motor retrocedía a unas 1,7 veces la velocidad con que avanzaba, así que se añadió al puente una escala para la marcha atrás, fijada en 0,75.

La media vuelta se escribió entonces como una maniobra propia del agente. Alterna marcha atrás y avance con la dirección a tope, siempre en el mismo sentido de giro. Cada tiempo se corta cuando la huella prevista del vehículo queda a menos de 0,10 m de un obstáculo, visto por el láser o marcado en el mapa. El láser no ve el borde de una escalera, así que en el mapa la escalera está cerrada. El primer tiempo va hacia el costado con más espacio, y el giro se mide con el rumbo del filtro.

El coordinador la pide antes de enviar una meta que queda a más de 110° del rumbo del vehículo y a más de 1,5 m. Si el agente no la completa en 120 s, sigue Nav2 como antes. La maniobra giró 160° frente a un salón del piso 3 y 167° junto a la escalera, en un solo tiempo cada vez. La cadena completa, del coordinador al agente y de ahí a Nav2, está verificada en las pruebas automáticas del coordinador, y en la campaña con los dos vehículos la maniobra se ejecutó 24 veces (sección 4.3).

3.5 Interfaz de interacción con el usuario

Esta sección corresponde al tercer objetivo específico: programar una interfaz móvil de interacción humano-robot basada en la selección del origen y el destino.

3.5.1 Una página web, no una aplicación

El objetivo pide «una interfaz móvil». Lo que se construyó es una página web que se abre en el navegador del teléfono. Cumple el objetivo, porque se usa desde el teléfono del usuario, y lo cumple mejor que una aplicación nativa para el usuario que el proyecto declara: un visitante ocasional no va a instalar nada para preguntar por un salón. La página debe funcionar además sin salida a internet, porque la red del edificio puede no tenerla, así que no usa bibliotecas ni recursos externos.

La página consta de un archivo HTML, una hoja de estilo y dos programas en JavaScript, 658 líneas en total. La sirve el propio computador portátil, y se comunica con ROS 2 por el puente *rosbridge*. Como el puente no tiene autenticación, el cortafuegos del portátil solo admite conexiones desde la red de los vehículos.

El primer programa es un cliente del protocolo del puente, escrito para este proyecto. La biblioteca de cliente más difundida no servía: implementa las acciones de la primera generación de ROS, que no son compatibles con las de ROS 2 (sección 2.3.7). El cliente propio envía la meta, recibe la retroalimentación y el resultado, y cancela, con las operaciones que el puente define para ROS 2. El segundo programa es la lógica de la página, y se comunica con el coordinador por cuatro canales. Recibe el catálogo de puntos de interés al conectarse y el estado de la misión una vez por segundo. Usa la acción de guiado para pedir y cancelar, y publica la confirmación de cambio de piso.

3.5.2 Lo que el usuario ve

El usuario elige el origen en una lista y el destino en un catálogo con búsqueda por nombre, y pulsa un botón. Cada destino muestra su piso y, si queda en el otro nivel, la marca «con relevo». A partir de ahí la página muestra el estado de la misión. Tiene un título calculado a partir de la etapa, el texto que redacta el coordinador, el vehículo que está guiando, hacia dónde va, el destino final y la distancia restante. Debajo, una franja de cinco pasos (preparando, tramo 1, relevo, tramo 2 y fin) marca el paso actual. La figura 3.8 es una misión en curso.

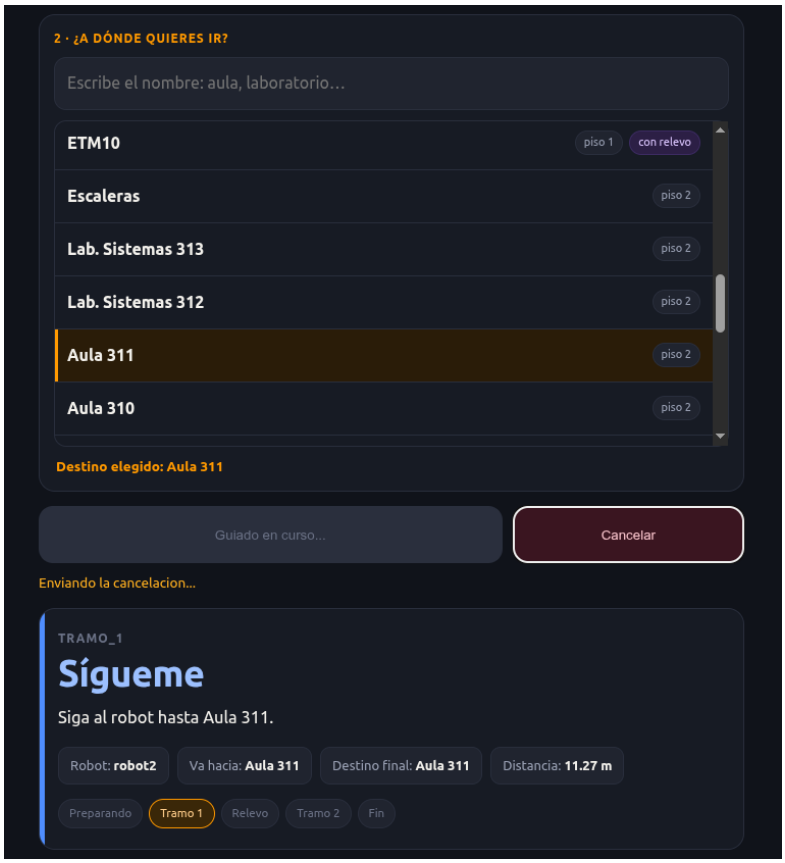


Figura 3.8. Interfaz durante una misión en simulación: el destino elegido, el botón de cancelar y el panel de estado del primer tramo, con el texto del coordinador, el vehículo activo y la distancia restante.

La interfaz se verificó contra el coordinador real: el catálogo, una misión con sus etapas, el texto del coordinador mostrado literal y el botón que se reactiva al terminar. También se operó una misión completa con relevo desde un teléfono real conectado a la red de los vehículos, sin tocar el computador.

Dos funciones se añadieron después de ver el sistema en uso, a pedido del director.

3.5.3 Confirmación de cambio de piso

En una misión entre niveles, el segundo vehículo llega a su escalera y espera al usuario. Pero el vehículo no puede saber si la persona ya subió. Sin una pausa explícita, la misión podía declararse completada con el usuario todavía en el piso equivocado, y la instrumentación lo habría registrado como éxito.

Por eso el coordinador pasa a la etapa de espera de confirmación cuando el segundo vehículo llega a

su punto de transferencia, y la página muestra entonces un botón para confirmar el cambio de piso (figura 3.9). El botón existe solo en esa etapa. A los 60 s sin confirmación la página muestra un aviso, y a los 120 s la misión termina como fallida. Los plazos se cuentan desde que llega el vehículo, para que un vehículo lento no le consuma tiempo al usuario, y se miden con el reloj del sistema y no con el de la simulación. Si el usuario confirma antes de que el vehículo llegue, la confirmación se retiene y se aplica al llegar.

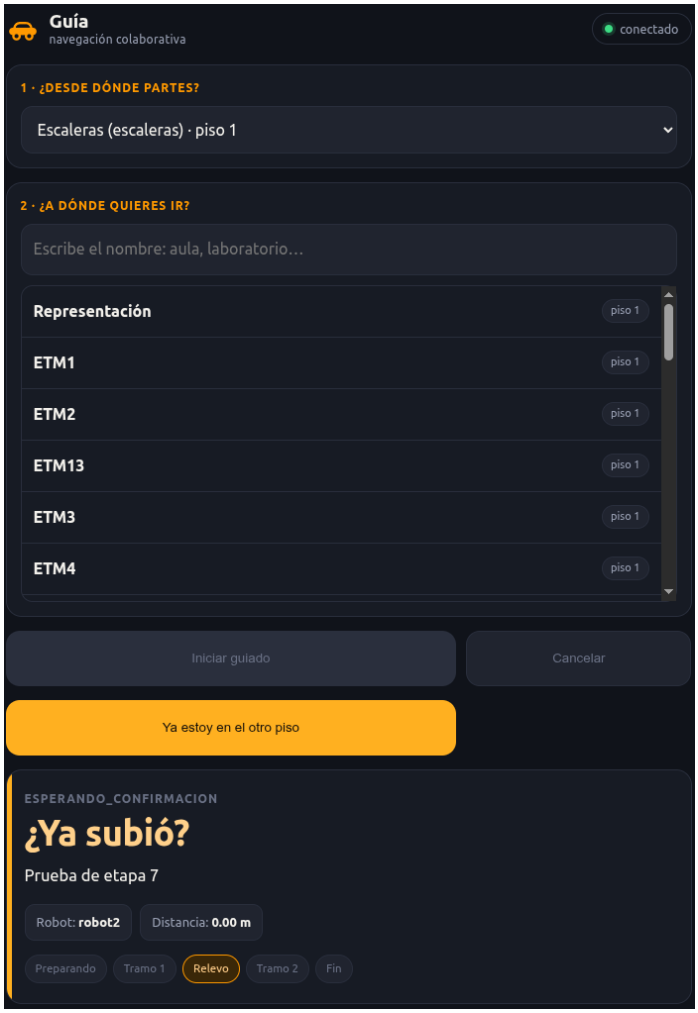


Figura 3.9. Interfaz en la etapa de espera de confirmación, durante su prueba automática: el estado de la misión lo publica la prueba, con el texto «Prueba de etapa 7», y la página muestra el botón «Ya estoy en el otro piso», de la versión anterior a los ajustes de la sección 3.5.5.

La función se verificó con tres corridas en simulación, una por cada camino. Confirmar a los pocos segundos termina la misión con éxito. Confirmar pasados los 60 s hace aparecer el aviso en el teléfono. Y no confirmar deja la misión fallida a los 120 s, con el motivo redactado.

3.5.4 Cancelación

El botón de cancelar existía pero no hacía nada, y no se había detectado. El servidor de acción del coordinador no registraba la función de respuesta a la cancelación, y ROS 2 rechaza por omisión, sin informar, toda cancelación que no tenga esa función. Al corregirlo se añadió el comportamiento que faltaba. Al cancelar, la misión pasa por la etapa de cancelación y el vehículo activo regresa al punto de transferencia de su piso; después la misión cierra como fallida, con un texto que lo dice. Se verificó con una corrida cancelada y una de control, y el vehículo quedó a 0,121 m de su punto de regreso.

Con los vehículos se cancelaron seis misiones, al menos una en cada etapa: yendo a recoger al usuario, en la transferencia, esperando la confirmación y en el segundo tramo. En las seis el vehículo volvió a su escalera, a entre 0,07 y 0,34 m de su punto, dentro de la tolerancia de 0,5 m. Esas pruebas mostraron un defecto que la simulación no podía mostrar, porque la media vuelta propia se añadió después. Nav2 conserva unos minutos las metas terminadas, y mientras tenía una cancelada en su lista el agente marcaba el vehículo como en error y rechazaba la media vuelta. Sin ella, Nav2 intentaba girar por su cuenta en el pasillo y oscilaba. Ahora la media vuelta solo espera a que no haya una meta en curso. Antes de la corrección se rechazaron ocho de quince medias vueltas, y después ninguna de diez.

3.5.5 Ajustes tras las pruebas con los vehículos

Las pruebas con los vehículos mostraron tres defectos de la página. El primero estaba en el título de la etapa de confirmación, que preguntaba siempre «¿Ya subió?». Al subir resultaba brusco, y al bajar del piso 4 al 3 era falso. Ahora el título sale de comparar el piso de origen con el del vehículo que espera, por ejemplo «Suba con calma al piso 4» o «Baje con calma al piso 3», y el botón dice «Ya estoy en el piso 4», o el piso que corresponda.

Ese cambio se hizo el mismo día de la campaña, antes de correrla, y sin embargo los teléfonos siguieron mostrando la versión anterior. El servidor de la página no le indicaba al navegador cuándo volver a pedirla, y el navegador usaba la copia que tenía guardada. La página se sirve ahora con la indicación de comprobarla cada vez que se abre.

El tercer defecto fue de nombres. Los dos ascensores se llamaban igual en la lista de destinos, y en una misión se eligió el del piso equivocado. Cuando un nombre se repite en los dos pisos, la página lo muestra ahora con su piso, como «Ascensor (piso 3)». Los tres ajustes se verificaron con pruebas automáticas de la página y del servidor, y después con un teléfono y los vehículos: en una misión de

subida y otra de bajada la página mostró el título que correspondía a cada sentido.

3.6 Protocolo experimental e instrumentación

Esta sección corresponde al cuarto objetivo específico: evaluar el desempeño del sistema con pruebas experimentales en un entorno interior controlado, con el tiempo de respuesta, el tiempo de asignación, la tasa de éxito y la continuidad del servicio entre niveles.

El protocolo se escribió antes de instrumentar el sistema, y esa secuencia es deliberada. Un protocolo redactado después de ver los datos tiende a acomodarse a lo que salió, por ejemplo eligiendo el umbral que deja fuera las corridas malas, y la campaña deja de poder refutar nada. El protocolo fija las métricas con su definición operativa, el criterio de éxito, el tamaño de muestra y una lista cerrada de causas de descarte, y cada versión queda fechada en el historial del repositorio.

3.6.1 Definiciones operativas

Una métrica sin un evento de inicio y uno de fin medibles por una máquina no se puede reportar. La tabla 3.7 da los dos eventos de cada una. Todas las marcas salen de un mismo reloj: en simulación, el de la simulación; en los vehículos, el reloj del computador que graba.

Tabla 3.7. Definición operativa de las cuatro métricas del anteproyecto.

Métrica	**Inicio**	**Fin**
Tiempo de respuesta	El coordinador acepta la solicitud	Primera de tres muestras seguidas de la odometría del vehículo asignado con velocidad de al menos 0,02 m/s
Tiempo de asignación	Primera publicación de la etapa recibida	Primera publicación del primer tramo, con el agente asignado
Tasa de éxito	Proporción de misiones exitosas sobre las corridas válidas	
Continuidad entre niveles	El primer vehículo alcanza su punto de transferencia	Primer movimiento del segundo vehículo

Una misión es exitosa si cumple tres condiciones. La pose final del vehículo, en su odometría, está a 0,25 m o menos del destino declarado en el catálogo. En los vehículos reales el umbral es de 0,5 m y el protocolo mide la llegada con flexómetro; en la campaña con los dos vehículos esa medida no se

tomó, y la llegada salió de la localización del vehículo (sección 4.4.4). La misión llegó a la etapa completada sin pasar por fallida. Y, si era entre niveles, hubo un relevo.

La regla principal del protocolo es que el éxito se mide contra la odometría del vehículo, nunca contra el resultado que devuelve Nav2. Una navegación puede terminar en éxito y haber dejado al vehículo lejos del destino: en las primeras pruebas en simulación, una de tres metas devolvió éxito con el vehículo a 0,296 m. Aceptar ese resultado sería dejar que el sistema se evalúe a sí mismo.

El tiempo de asignación resultó demasiado pequeño para el reloj de la simulación, que avanza en saltos de 100 ms. Por eso se reporta como una cota por misión y se caracterizó aparte, en un banco de pruebas que corre el coordinador aislado con el reloj del sistema. La continuidad entre niveles se definió primero como una condición binaria, que resultó no poder fallar, y se reformuló como se explica en la sección 4.4.

3.6.2 Diseño de la campaña en simulación

La campaña en simulación tiene treinta misiones: quince con origen y destino en el mismo piso, que sirven de control, y quince entre pisos, que llevan un relevo y son las que responden la pregunta de investigación. Con diez misiones corridas se vio que las misiones entre pisos navegaban siempre por el piso 2 y las de control no. Un resultado peor en las primeras no habría permitido distinguir el efecto del relevo del efecto del piso. El diseño se enmendó para equilibrar el piso dentro de cada grupo, en cuatro estratos de ocho, siete, ocho y siete misiones. Las diez misiones ya corridas se conservaron y la enmienda quedó escrita en el protocolo.

Los pares de origen y destino se sortean con una semilla fija, publicada antes de correr. Con pares elegidos a mano se corre el riesgo de probar justo los recorridos que funcionan. Las misiones se ejecutan en el orden del sorteo.

Una corrida fallida cuenta como fallo. Solo se descarta si la causa es ajena al sistema medido y está en una lista cerrada, escrita antes de la campaña. Las causas admitidas son la caída del simulador, un controlador del vehículo que no se activa, el factor de tiempo real por debajo de 0,99, un fallo de la máquina anfitriona y una cancelación por una causa externa anotada antes de conocer el resultado. Si los descartes superan el 20 % de las corridas, la campaña no es válida.

3.6.3 Instrumentación

Cada misión deja una grabación de datos de ROS (*rosbag*) con los tópicos del coordinador y de los dos vehículos. A partir de esa grabación, un programa compone el registro de la misión, en un formato estructurado que se valida contra un esquema versionado. El registro contiene las marcas de cada etapa, la trayectoria de cada vehículo, la pose final, el error de llegada y su procedencia. Un segundo programa lee los registros de la campaña y calcula las cuatro métricas con sus intervalos de confianza. Antes de calcular, comprueba la integridad de cada registro, y si alguno contradice las reglas de descarte no lo cuenta en ningún sitio, de modo que el total deja de cuadrar y el error salta a la vista.

El analizador se escribió antes de correr la campaña y encontró dos defectos que después habrían sido irreparables. La métrica de continuidad no tenía campo en el esquema, y el intervalo de confianza que citaba el protocolo no se reproducía con el método declarado.

Para los vehículos reales la cadena tiene dos diferencias. El grabador corre en el contenedor con Jazzy, así que un programa adapta la grabación al formato que leen las herramientas de Humble. Y el registro usa el reloj del computador, la tolerancia de 0,5 m y el mapa de cada piso. La distancia de llegada se guarda con su procedencia: medida con flexómetro o, si no la hay, tomada de la localización del vehículo, sin incertidumbre asignada. El analizador cuenta las llegadas de cada procedencia y advierte cuando no hay medida externa.

3.6.4 Pruebas de verificación de la demostración física

La demostración sobre los vehículos se organizó en seis pruebas de verificación sucesivas, con criterios propuestos por los autores y decididos por los directores. Cada prueba tiene un criterio escrito antes de correrla y lo que significa no superarla (tabla 3.8). Se fijaron tres fechas de revisión en octubre, y después de la última, el 16 de octubre, no se toman más datos.

Tabla 3.8. Pruebas de verificación de la demostración física y su estado al cerrar la toma de datos.

Prueba	**Criterio**	**Estado**
Actuación	El vehículo responde a las órdenes de dirección y tracción	Superada
Odometría	Error de 10 % o menos sobre un recorrido conocido de 5 m o más	Superada
Navegación de un vehículo	Un vehículo llega de un punto a otro con Nav2, con la llegada verificada	Superada
Dos vehículos en la misma red	Los dos vehículos y el coordinador coexisten sin conflicto de nombres	Superada
Misión con relevo	Una misión con relevo sobre los dos vehículos, con su registro	Superada
Campaña con los vehículos	Entre 5 y 10 repeticiones del protocolo con su registro	Superada

Para la campaña con los vehículos se sortearon, antes de correr y con una semilla fija, diez misiones entre los pisos 3 y 4: cinco de subida y cinco de bajada. Se corrieron en ese orden y sin reubicar los vehículos entre misiones, como ocurriría en la operación real. Sus resultados están en la sección 4.3.

4. RESULTADOS

Los resultados llegan de tres fuentes. La primera es la campaña en simulación, con treinta misiones sorteadas (sección 4.1). La segunda son las pruebas de verificación de la plataforma física, que miden la odometría y la navegación de un vehículo (sección 4.2). La tercera es el sistema completo sobre los dos vehículos: una misión con relevo y una campaña de misiones sorteadas entre los pisos 3 y 4 (sección 4.3). Las dos campañas se reportan por separado. La de simulación produce la evidencia estadística, y la de los vehículos demuestra que el sistema opera sobre la plataforma real.

4.1 Evaluación en simulación

La campaña corrió treinta misiones sorteadas: quince de control, con origen y destino en el mismo piso, y quince entre pisos, con un relevo. El analizador la dictaminó válida, sin ningún descarte, frente a un máximo admitido del 20 %.

4.1.1 Tasa de éxito

Una misión es exitosa si cumple dos condiciones. La primera, que la pose final quede a 0,25 m o menos del destino, medida contra la odometría y no contra el resultado de Nav2. La segunda, que la misión llegue a completarse sin pasar por fallo. La tabla 4.1 da la tasa de cada grupo con su intervalo de confianza de Wilson al 95 % [53].

Tabla 4.1. Tasa de éxito de la campaña en simulación, con intervalo de confianza de Wilson al 95 %.

Misiones	**n**	**éxitos**	**tasa**	**intervalo al 95 %**
Campaña completa	30	26	86,7 %	70,3 – 94,7 %
Mismo piso (control)	15	12	80,0 %	54,8 – 93,0 %
Entre pisos, con relevo	15	14	93,3 %	70,2 – 98,8 %

Lo que se puede afirmar es que la tasa de éxito supera el 70 %, no que sea del 86,7 %. Con treinta misiones el intervalo mide 24 puntos porcentuales de ancho, y ese límite es de diseño: solo se reduce con más muestra. El 86,7 % es el estimador puntual y se reporta siempre junto a su intervalo.

Los intervalos de los dos grupos se solapan, así que la comparación entre ellos no sostiene ninguna conclusión. Conviene decirlo porque el estimador puntual apunta en contra de la intuición: las misiones con relevo salieron mejor que las de control.

4.1.2 Causa común de los cuatro fallos

Las cuatro misiones fallidas quedaron a 0,295, 0,284, 0,311 y 0,347 m del destino, frente a una tolerancia de 0,25 m. Las cuatro se quedaron cortas: ninguna se pasó del destino ni terminó en otro sitio. Los cuatro fallos se deben al límite de precisión de la aproximación final, y ninguno a la coordinación. El error de llegada del conjunto tiene una mediana de 0,106 m y va de 0,019 a 0,347 m.

4.1.3 Tiempo de respuesta y tiempo de asignación

El tiempo de respuesta se reporta por escalones, y no con media y desviación, por una razón de instrumento (tabla 4.2).

Tabla 4.2. Tiempo de respuesta de las treinta misiones, en escalones del reloj de la simulación.

Tiempo de respuesta	0,1 s	0,2 s	0,3 s	0,4 s
Misiones	10	15	4	1

Los cuatro valores son múltiplos exactos de 100 ms, que es el paso con que avanza el reloj de la simulación. La resolución del instrumento vale entre el 25 % y el 100 % de la magnitud medida. Una mediana de 0,2 s significa «uno o dos pasos del reloj», no «doscientos milisegundos», y dar media y desviación sugeriría una precisión que el reloj no entrega. Lo que sí se sostiene es la cota, que además es lo que le importa a un usuario: el robot arranca en menos de medio segundo.

El tiempo de asignación quedó por debajo de 100 ms en las treinta misiones, varios órdenes de magnitud por debajo de un paso del reloj. Por eso se caracterizó aparte, en un banco de pruebas con el reloj del sistema: mediana entre 154 y 175 µs y máximo de 306 µs.

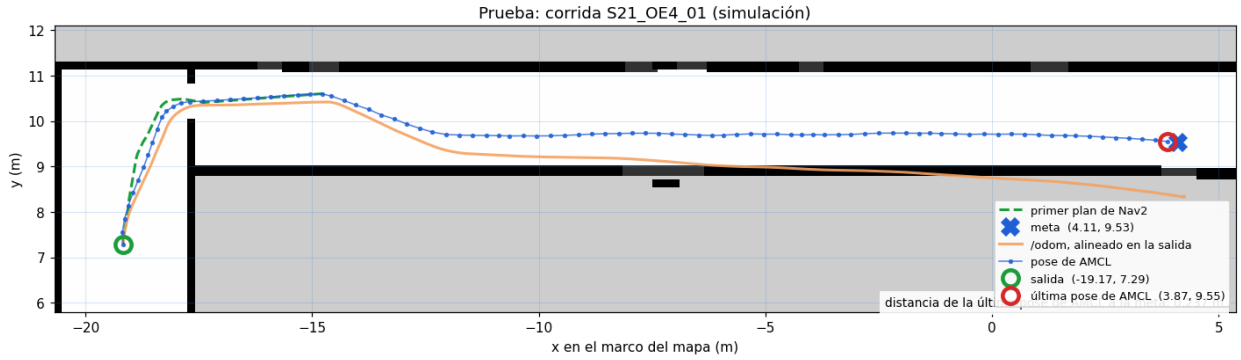


Figura 4.1. Una misión de la campaña en simulación. Se superponen el primer plan que calcula el navegador, la pose estimada a lo largo del recorrido y la odometría cruda. La separación creciente entre las dos últimas es la deriva que la localización corrige contra el mapa.

4.2 Evaluación con los vehículos

La demostración física se organizó en pruebas de verificación sucesivas (sección 3.6.4). Esta sección reporta las que se han superado y que producen cifras: la de odometría y la de navegación de un vehículo.

4.2.1 La odometría del vehículo

El vehículo no tiene codificadores en las ruedas, así que su odometría sale de comparar barridos sucesivos del láser. Durante meses esa pieza no produjo una medida utilizable. La tabla 4.3 reúne lo que se midió en cada entorno, como fracción del avance real medido con flexómetro.

Tabla 4.3. Fracción del avance real que registra la odometría, por entorno. El criterio de la prueba es un error de 10 % o menos.

Entorno	Recorrido	Registrado	Criterio
Pasillo simulado, 46,9 m	largo	37,9 %	no cumple
Pasillos de los pisos 1 y 2	largo	5,7 % y 1,3 %	no cumple
Hall del piso 2	6 m	73 %	no cumple
Cuarto cerrado, desplazado a mano	3 m	96,3 – 101,9 %	cumple
Pasillo del piso 4	6,68 m	95,4 %	cumple
Pasillo del piso 4	14,57 m	103,3 %	cumple
Pasillo del piso 4, con la unidad inercial	8,08 m	101,9 %	cumple
Pasillo del piso 4, con la unidad inercial	2,82 m	104,9 %	cumple

El estimador es el mismo en todas las filas; lo que cambió fue el entorno. En un pasillo recto y uniforme la información de avance no está en la señal del sensor, y ningún ajuste la recupera (sección 2.3.5). En un pasillo con hornacinas, casilleros y quiebres, con ocho cambios de perfil en 25 m, la misma odometría queda dentro del ± 5 % sin cambiar un parámetro.

Con las dos mediciones del pasillo del piso 4, de 6,68 y 14,57 m, los directores dieron por superada la prueba de odometría. Era la condición que el proyecto venía señalando como el paso del que dependía todo lo demás sobre los vehículos.

Las dos últimas filas ya incluyen la unidad inercial (sección 3.4.8). El avance lo estima el filtro, que lo toma de la misma odometría del láser. El segundo tramo mide 2,82 m, por debajo de los 5 m que pide el criterio, y se incluye como dato de la misma cadena.

4.2.2 La navegación y la precisión de llegada

La prueba de navegación de un vehículo pide un recorrido de punto a punto con Nav2 y la llegada verificada con flexómetro, a 0,5 m o menos del destino. Esa es la tolerancia que fijó el director para los vehículos reales.

En la primera sesión en el piso 4, las dos llegadas medidas quedaron a 0,57 y 0,62 m, cortas. Nav2 daba entonces la meta por alcanzada a 1,0 m, y los dos vehículos se detuvieron al cruzar ese margen. Con la unidad inercial integrada (sección 3.4.8) el margen bajó a 0,5 m. En la sesión siguiente, uno de los vehículos recorrió en el piso 4 tres tramos encadenados, sin que nadie lo tocara entre uno y otro, y después regresó a la salida (tabla 4.4 y figura 4.2).

Tabla 4.4. Cadena de tres tramos en el piso 4 y regreso a la salida: llegada de cada tramo.

Tramo	Destino	Tiempo	Recuperaciones	A la meta	Medida
1	Salón 403	13,5 s	1	0,26 m	estimada
2	Salón 402	27,8 s	1	0,45 m	flexómetro
3	Salón 401	5,2 s	0	0,06 m	flexómetro
Regreso	Salida	62,8 s	6	0,34 m	flexómetro

Las llegadas medidas quedaron dentro de la tolerancia de 0,5 m, y los directores dieron por superada la prueba. La llegada al Salón 403 no se midió porque en ese momento no había flexómetro, y se estimó con el error de los otros dos tramos. El regreso a la salida incluyó una media vuelta en el tramo ancho frente a los salones, que Nav2 resolvió con seis recuperaciones.

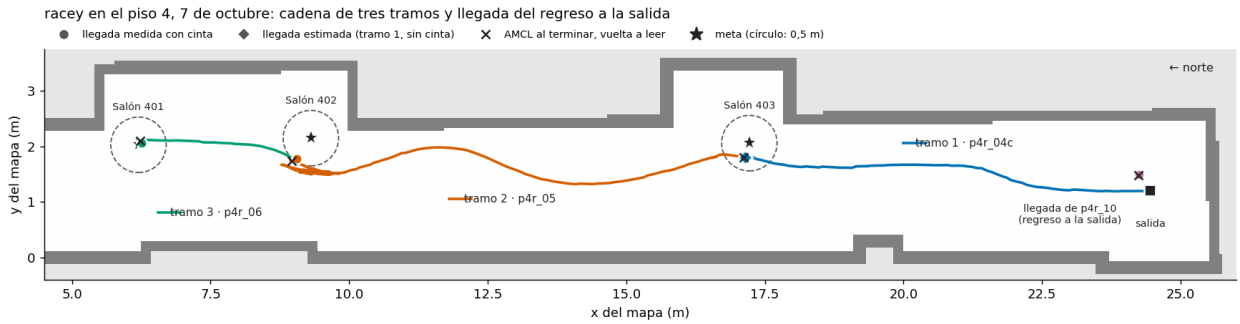


Figura 4.2. Cadena de tres tramos en el piso 4 y regreso a la salida: recorrido según la localización, llegadas medidas con flexómetro, la llegada estimada al Salón 403 y la pose de la localización al terminar cada tramo.

Al día siguiente, el otro vehículo recorrió el piso 3 en cinco tramos encadenados, con dos medias vueltas entre ellos, y los cinco terminaron con éxito y sin recuperaciones. Esas llegadas no se midieron con flexómetro. Al terminar cada tramo se forzó a la localización a recalcular su pose, y esta se desplazó entre 0,61 y 0,86 m. La pose que publica la localización al detenerse el vehículo va atrasada, y por eso la llegada que vale es la medida con flexómetro.

Lo que condiciona la precisión de llegada sigue siendo la banda muerta de tracción (sección 3.4.7): el vehículo se aproxima a la meta a 0,40 m/s y no tiene régimen de aproximación fina.

4.3 Evaluación del sistema completo con los dos vehículos

El sistema completo funcionó sobre los dos vehículos. La misión con relevo del Salón 302 al Salón 402 se completó, y de las diez misiones sorteadas para la campaña, las nueve que llegaron a correr se completaron, todas con relevo. La décima quedó interrumpida porque el computador portátil se apagó por falta de batería. En las nueve, el vehículo del piso de destino terminó a menos de 0,5 m del destino según su propia localización, que es la medida con que se verificó la llegada (sección 4.4.4).

4.3.1 Condiciones de las pruebas

Las pruebas se hicieron en los pisos 3 y 4, con un vehículo en cada piso. El coordinador y el grabador corrieron en el computador portátil, y cada misión se pidió desde la página web en un teléfono conectado a la red de los vehículos. Los autores hicieron de usuario: pidieron cada misión, acompañaron a los vehículos y confirmaron el cambio de piso en la página.

Los vehículos no se reubicaron entre misiones. Cada misión empezó donde había terminado la anterior, y el vehículo del piso de origen fue primero a recoger al usuario, como ocurriría en la operación real. Cuando el destino quedaba detrás del vehículo, el coordinador pidió la media vuelta en varios tiempos (sección 3.4.10).

La llegada no se midió con flexómetro. El error de llegada que se reporta es la distancia entre la pose del vehículo según su localización, leída 1,5 s después de que Nav2 diera la meta por alcanzada, y el punto del catálogo. Es la misma medida con que el coordinador acepta o rechaza cada llegada.

4.3.2 La misión con relevo

La primera prueba del sistema completo fue una misión del Salón 302, en el piso 3, al Salón 402, en el piso 4. Se completó en 132,5 s, con un relevo y con la llegada a 0,444 m del destino. Entre la llegada del primer vehículo a la escalera y el arranque del segundo pasaron 61,1 s. Fue el segundo intento: en el primero, el coordinador rechazó la llegada del vehículo del piso 3 al Salón 302, adonde iba a recoger al usuario, porque su localización lo situaba a 0,59 m del punto.

Esa sesión sirvió de ensayo del montaje, y en ella los autores eligieron las misiones sin seguir el sorteo. De las doce misiones grabadas, siete se completaron y cinco terminaron fallidas. En cuatro, el coordinador rechazó una llegada con el vehículo a entre 0,51 y 0,78 m del punto, y en la otra Nav2 abortó la navegación. Estas misiones no forman parte de la campaña.

4.3.3 La campaña con las misiones sorteadas

Las diez misiones de la campaña se sortearon antes de correr, con una semilla fija, entre los puntos de interés de los dos pisos: cinco de subida y cinco de bajada (sección 3.6.4). Corrieron en el orden del sorteo. La tabla 4.5 da el resultado de las nueve que llegaron a correr.

Tabla 4.5. Campaña con los dos vehículos: las nueve misiones sorteadas que corrieron, todas completadas con un relevo. La llegada es la de la localización del vehículo, con una tolerancia de 0,5 m.

N.º	Origen	Destino	Tiempo total	Respuesta	Llegada
1	Ascensor, piso 3	Salón 403	52,3 s	0,51 s	0,435 m
2	Salón 302	Salón 401	90,1 s	0,27 s	0,202 m
3	Salón 401	Salón 302	87,6 s	3,90 s	0,143 m
4	Salón 403	Salón 303	78,9 s	1,25 s	0,420 m
5	Salón 402	Salón 301	105,4 s	0,66 s	0,285 m
6	Salón 301	Salón 401	89,8 s	2,97 s	0,226 m
7	Ascensor, piso 4	Salón 301	81,7 s	0,87 s	0,234 m
8	Salón 302	Salón 403	81,8 s	0,95 s	0,214 m
9	Salón 303	Salón 402	92,6 s	0,63 s	0,299 m
Mediana			87,6 s	0,87 s	0,234 m

Con nueve éxitos en nueve misiones, el intervalo de Wilson al 95 % va del 70,1 al 100 %. Lo que se puede afirmar es que la tasa de éxito sobre los vehículos supera el 70 %, la misma cota que sostiene la campaña en simulación. Expresado como en la sección 2.3.8, nueve repeticiones sin fallo demuestran una fiabilidad mínima de 0,81 con un 85 % de confianza.

El error de llegada tuvo una mediana de 0,234 m y fue de 0,143 a 0,435 m. El tiempo total, desde la solicitud hasta la llegada verificada, tuvo una mediana de 87,6 s y fue de 52,3 a 105,4 s.

El tiempo de respuesta, desde que el coordinador acepta la solicitud hasta que el vehículo asignado empieza a moverse, tuvo una mediana de 0,87 s y quedó por debajo de 1,3 s en siete de las nueve misiones. Los dos valores más altos, 3,90 y 2,97 s, son de misiones en que el vehículo ya estaba en el punto de origen. El coordinador verificó esa llegada, que no exigía moverse, con la espera de 1,5 s, y el primer movimiento fue la media vuelta hacia la escalera.

El tiempo de asignación fue de 0,3 ms en las nueve misiones, medido con el reloj del portátil. La cifra incluye el paso del mensaje de estado hasta el grabador, y es del mismo orden que la del banco de pruebas de la sección 4.1.

En las nueve misiones los vehículos hicieron 24 medias vueltas, de entre 160 y 171°, en uno a cuatro tiempos cada una.

La figura 4.3 descompone el tiempo de cada misión en sus etapas. Ir a recoger al usuario tomó entre 2,2 y 30,6 s, según dónde había terminado la misión anterior. El guiado en el piso de origen tuvo una mediana de 24,2 s, y el guiado en el piso de destino, de 24,8 s.

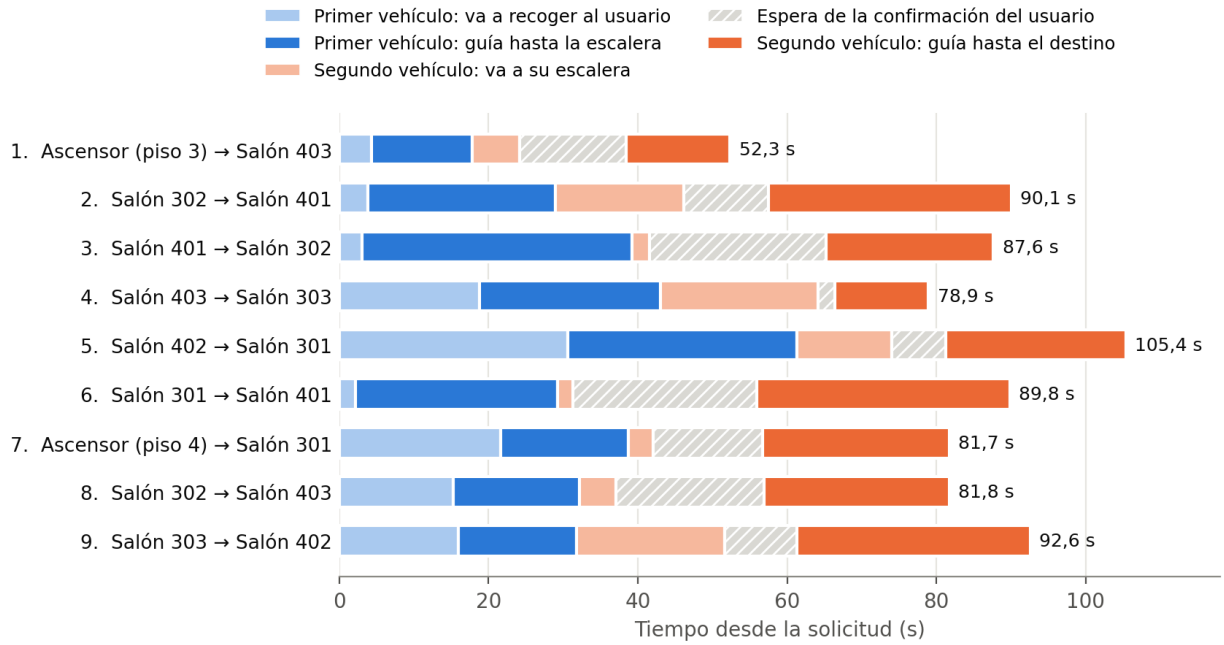


Figura 4.3. Duración de cada etapa en las nueve misiones de la campaña con los vehículos, según las marcas de etapa del coordinador. El tramo rayado es la espera de la confirmación de cambio de piso, contada desde que el segundo vehículo está en su escalera.

4.3.4 El relevo entre los dos vehículos

En las nueve misiones el relevo se produjo y el coordinador no pasó por el estado inactivo ni quedó sin agente asignado. Como en simulación, ese criterio no puede fallar por la forma en que está construido (sección 4.4). La medida que informa es el intervalo del relevo: el tiempo desde que el primer vehículo llega a su escalera hasta el primer movimiento del segundo.

Ese intervalo tuvo una mediana de 1,22 s, pero la cifra junta dos situaciones. En cinco misiones el segundo vehículo tuvo que desplazarse hasta su escalera, en cuatro desde el destino de la misión anterior y en una para corregir su posición unos centímetros, y arrancó en menos de 1,3 s. En las otras cuatro ya estaba en la escalera y no tuvo que moverse. Su primer movimiento fue entonces el del guiado final, después de la confirmación, y el intervalo, de 12,9 a 27,5 s, mide sobre todo cuánto

tardó la persona en confirmar.

Para la persona, lo que cuenta es el tiempo entre la llegada a la escalera del piso de origen y el arranque del guiado en el piso de destino. En ese tiempo la persona cambia de piso, el segundo vehículo llega a su escalera si no estaba allí y la persona confirma en la página. Tuvo una mediana de 24,8 s y fue de 18,0 a 29,5 s. La confirmación, desde que el segundo vehículo estaba en su escalera, tomó entre 2,3 y 24,7 s, y el aviso de los 60 s no llegó a mostrarse en ninguna misión.

4.3.5 Criterio de conteo e incidencias

La campaña cuenta, por decisión de los autores, una corrida por misión: la que se completó. El protocolo, en cambio, cuenta como fallo toda corrida fallida, y por eso los intentos que no entran en la tasa se declaran aquí. Las misiones 1 y 2 se completaron en el segundo intento. Los dos primeros intentos fallaron en el tramo en que el vehículo del piso 3 iba a recoger al usuario. En el de la misión 1, el coordinador rechazó la llegada al ascensor, con el vehículo a 0,526 m del punto, y en el de la misión 2 no tenía la pose del vehículo para verificar la llegada.

Antes de esos intentos, la navegación de los dos vehículos dejó de responder. Ocho intentos de la misión 1 terminaron rechazados o abortados, con las transformadas de posición detenidas dentro de Nav2. Los dos vehículos llevaban horas encendidos, con cargas de 10 a 14 en procesadores de dos núcleos. Se reiniciaron las cadenas de navegación con la pose de salida, y después las nueve misiones corrieron sin tocar los vehículos. La causa de la detención no se estableció.

La misión 10 empezó y no terminó: el portátil se apagó por falta de batería unos 30 s después de pedirla, cuando el vehículo del piso 4 ya había llegado a su escalera. Unos minutos antes se había pedido por error el ascensor del piso 4 como destino, porque los ascensores de los dos pisos aparecían en la página con el mismo nombre. Ese intento no es una misión del sorteo y no se cuenta. Después de la campaña, la página muestra el piso junto a los nombres que se repiten (sección 3.5.5).

4.4 Limitaciones de la medición

Cuatro límites afectan a cómo deben leerse los resultados anteriores. Los tres primeros son de la campaña en simulación y el cuarto, de la campaña con los vehículos. Se declaran aquí porque callarlos sería presentar como medida lo que no lo es.

4.4.1 Naturaleza del criterio de continuidad entre niveles

La continuidad del servicio es la variable de respuesta principal del cuarto objetivo, y su criterio binario se cumplió en las treinta misiones. Ese resultado, sin embargo, no es evidencia, y la razón es estructural.

El criterio exige que, dentro de la ventana de la misión, el coordinador nunca esté en estado inactivo y nunca tenga el agente sin asignar. Los dos únicos puntos del código que producen esos valores ocurren antes de que la ventana se abra. No existe ejecución posible que lo incumpla: son cero incumplimientos de cero posibles. Un criterio que no puede dar resultado negativo no aporta evidencia, por muchas veces que se verifique [41].

La corrección no consistió en inventar una ruta de fallo, que habría sido instrumentar el experimento para que pareciera más informativo. La continuidad se declaró como un invariante estructural del diseño. La afirmación se apoyó en una magnitud que sí varía: el intervalo de tiempo del relevo, entre el final del tramo de un vehículo y el inicio del tramo del otro. En las quince misiones entre pisos tuvo una mediana de 0,1 s y un máximo de 0,2 s, es decir, uno o dos pasos del reloj de la simulación.

4.4.2 Ancho del intervalo de confianza

Ya se dijo con la tabla 4.1, y se repite porque es el límite más fácil de olvidar al redactar: con treinta misiones, lo que se puede defender es una cota inferior, no un valor puntual. Aumentar la confianza exige más muestra.

4.4.3 Espacio desconocido marcado como libre en el mapa

El mapa con que corrió la campaña declaraba un umbral que hacía que 10 908 celdas desconocidas, el 14,5 % del mapa, se cargaran como espacio libre. Con ello quedaban desactivadas en la práctica las dos salvaguardas que debían impedir planificar sobre lo no explorado.

Esto no invalida el veredicto de la campaña. Los cuatro fallos fueron de precisión de llegada, no de trazado, y el pasillo por el que se navegó está mapeado como libre en el mundo real. Lo que no se sabe es si alguna trayectoria cruzó una de esas celdas, y es comprobable sobre los registros conservados. El defecto se corrigió después de la campaña.

4.4.4 La llegada en los vehículos, sin medida externa

En la campaña con los vehículos la llegada no tuvo una medida independiente del sistema. El protocolo la toma con flexómetro sobre marcas en el piso, y en esa sesión no se midió. El error que se reporta es el de la localización del propio vehículo contra el punto del catálogo, leído 1,5 s después de que Nav2 diera la meta por alcanzada, con el vehículo ya detenido. Es la medida con que el coordinador acepta la llegada. En esta campaña, por tanto, el sistema verificó su llegada con su propia estimación, que es lo que la regla principal del protocolo procura evitar (sección 3.6). Los registros de las misiones lo declaran con la procedencia de la medida, y el analizador lo advierte al calcular las métricas.

La única comparación entre la localización y el flexómetro en los vehículos es la de la cadena de tres tramos en el piso 4 (sección 4.2.2). Leída de nuevo al terminar cada tramo, la localización quedó a 0,10 m o menos de la medida en el sentido del avance y a 5 cm o menos de lado. En el piso 3, en cambio, forzar a la localización a recalcular su pose al final de cada tramo la desplazó entre 0,61 y 0,86 m. El resultado de la campaña se lee entonces así: cada vehículo, según su propia localización, llegó a menos de 0,5 m del destino. Que la distancia real cumpliera la misma tolerancia no está medido.

CONCLUSIONES

El primer objetivo específico, modelar la arquitectura funcional del sistema, se cumplió. El sistema tiene un contrato de interfaces escrito, treinta y seis requisitos con su prueba, y una capa de coordinación que ejecuta el relevo entre niveles. Escribir el planificador como función pura, sin depender de la capa de comunicaciones de ROS 2, resultó más valioso de lo previsto. Permite verificar las 930 combinaciones de origen y destino en milisegundos, y encontró defectos que una corrida en el simulador no habría delatado, como planes que mandaban al mismo robot dos veces a la misma pose. La asignación de tareas es óptima por la estructura del problema, con un solo agente admisible por nivel, y el aporte del trabajo está en coordinar dos agendas que dependen una de otra durante el relevo.

El segundo objetivo, la plataforma robótica de dos vehículos, se cumplió. Los dos vehículos están integrados, y Nav2 quedó configurado para cinemática tipo automóvil, con el planificador, el controlador y el árbol de comportamiento sustituidos por los que esa cinemática admite (sección 3.4.3). La odometría cumple su criterio, con un error entre $-4{,}6\,\%$ y $+3{,}3\,\%$ sobre recorridos de 6,68 y 14,57 m, y los directores dieron por superada la prueba correspondiente. La navegación de un vehículo con la llegada verificada también se superó. En una cadena de tres tramos sin intervención, las llegadas medidas con flexómetro quedaron entre 0,06 y 0,45 m de la meta, dentro de la tolerancia de 0,5 m (sección 4.2.2). Los dos vehículos operan juntos en la misma red, con un tiempo de ida y vuelta de los mensajes de 5,99 ms de mediana. En la campaña completaron nueve misiones seguidas con relevo sin que se los reubicara, con 24 medias vueltas en pasillos de 2,5 a 3,2 m de ancho.

Quedan dos límites en la plataforma. El primero es la banda muerta de tracción (sección 3.4.7). El vehículo no avanza por debajo de 0,40 m/s y no tiene régimen de aproximación fina, y por eso la tolerancia de llegada con los vehículos es de 0,5 m y no de 0,25 m, como en simulación. El segundo es la estabilidad en sesiones largas: tras horas encendidos y con el procesador muy cargado, la navegación de los dos vehículos dejó de responder y hubo que reiniciarla, por una causa que no se estableció.

El tercer objetivo, la interfaz de interacción con el usuario, se cumplió. La interfaz funciona desde el navegador de un teléfono, sin instalación y sin salida a internet, y se operó con ella una misión completa con relevo desde un dispositivo real. Dos funciones se añadieron después de ver el sistema en uso. Una es la confirmación de cambio de piso, que impide dar por completada una misión con el usuario en el nivel equivocado. La otra es la cancelación, que no funcionaba porque el servidor

de acción rechazaba las solicitudes sin informar. Con la interfaz se pidieron y siguieron todas las misiones de la campaña con los vehículos, y la confirmación de cambio de piso se usó en cada relevo. Esas pruebas mostraron defectos de la página que ya están corregidos (sección 3.5.5). El título de la confirmación no indicaba si el usuario sube o baja, dos destinos de pisos distintos tenían el mismo nombre y el teléfono conservaba una versión anterior de la página.

El cuarto objetivo, la evaluación experimental, se cumplió en simulación y con los vehículos. La campaña de treinta misiones sorteadas en simulación fue válida, sin descartes, y permite afirmar con 95 % de confianza que la tasa de éxito supera el 70 %. Con los dos vehículos, en los pisos 3 y 4, las nueve misiones sorteadas que llegaron a correr se completaron con relevo, lo que sostiene la misma cota (sección 4.3). Su tiempo total tuvo una mediana de 87,6 s, el tiempo de respuesta, de 0,87 s, y el de asignación fue de 0,3 ms. La llegada de esa campaña se verificó con la localización de cada vehículo y no con una medida externa (sección 4.4.4). Lo demostrado es que cada vehículo, según su propia estimación, terminó a menos de 0,5 m del destino. Las misiones 1 y 2 se completaron al segundo intento, y esos primeros intentos se declaran fuera de la tasa.

Sobre el objetivo general, el esquema de relevo es viable. Dos robots dedicados cada uno a un nivel pueden repartirse el guiado de una persona que cambia de piso, sin que ninguno atraviese la zona de transición vertical y sin modificar el edificio. Eso quedó demostrado en simulación, con una campaña medida y válida, y sobre los dos vehículos reales, que ejecutaron el relevo entre los pisos 3 y 4 en cada una de las nueve misiones de su campaña. La odometría y la navegación de los vehículos, que durante meses hicieron dudar de que el esquema fuera ejecutable fuera de la simulación, dejaron de ser el límite. Lo que el trabajo no mide es la experiencia de la persona guiada, que queda para un estudio con usuarios.

Queda una conclusión de método. En este proyecto costó más averiguar qué medía cada prueba que construir el sistema, como muestran tres casos. El tiempo de asignación valía cero en todas las grabaciones, porque el reloj de la simulación no tenía resolución para medirlo, hasta que se caracterizó en un banco aparte. El criterio de continuidad entre niveles no podía dar un resultado negativo, así que sus treinta verificaciones no eran evidencia. Y la correspondencia entre el modelo del edificio y el edificio real se daba por supuesta, y resultó falsa en un 20 % la primera vez que se midió. Los tres se encontraron preguntando si el criterio podía fallar, y ninguno habría aparecido corriendo más experimentos. La campaña con los vehículos añadió un caso más. El intervalo del relevo, que en simulación medía la reacción del sistema, en los vehículos medía a veces cuánto tardaba la persona en confirmar, y solo se vio al descomponer cada misión en sus etapas.

TRABAJOS FUTUROS

Las líneas que siguen salen de límites concretos que el trabajo encontró y documentó, y de las suposiciones que el diseño hizo para mantenerse dentro del alcance.

Aproximación fina del vehículo. Es la limitación que condiciona la precisión de llegada, y está localizada y medida (sección 3.4.7). El vehículo no se mueve por debajo de 0,40 m/s, porque por debajo de ese valor la cadena de tracción no entrega potencia suficiente. Hay dos vías para resolverla. Una es de hardware: calibrar el variador o cambiar la relación de transmisión, para darle al vehículo un régimen lento del que hoy carece. La otra es de control: cerrar el lazo de velocidad con una medida de avance, de modo que el vehículo pueda regularla cerca de la meta.

Una fuente de avance independiente del láser. La unidad inercial resolvió el rumbo, pero la posición sigue saliendo de comparar barridos del láser, y eso falla por construcción en pasillos uniformes (sección 2.3.5). El proyecto lo resolvió eligiendo un entorno con estructura, y esa solución limita dónde puede operar el sistema. Codificadores en las ruedas, u odometría visual con las cámaras que los vehículos ya llevan, darían información de avance que no depende de la geometría del sitio.

Una medida externa de la llegada. En la campaña con los dos vehículos la llegada se verificó con la localización del propio vehículo (sección 4.4.4). Repetirla con una medida independiente daría la distancia real a cada destino. Puede ser el flexómetro sobre marcas en el piso, como pedía el protocolo, o un sistema de seguimiento externo, que además registraría la trayectoria completa sin depender del vehículo.

Estabilidad de la navegación en sesiones largas. Tras horas encendidos, la navegación de los dos vehículos dejó de responder y hubo que reiniciarla (sección 4.3). La causa no se estableció. Antes de dejar el sistema en servicio hace falta reproducir la falla en una sesión larga controlada. Con ella se puede decidir si basta con supervisar la cadena de navegación y reiniciarla sola, o si hay que bajar la carga del procesador del vehículo.

El ascensor como segunda vía de transición. Hoy el cambio de piso se hace siempre por la escalera, y el ascensor es un destino más. Admitir los dos exige que el planificador elija entre dos

puntos de transferencia por nivel, presumiblemente el más cercano al usuario o el que este prefiera. El planificador ya detecta ese caso y falla con un mensaje explícito cuando encuentra dos, de modo que el punto de extensión está identificado.

Esquinas en pasillos estrechos. Todos los recorridos medidos fueron sobre tramos rectos. La media vuelta en varios tiempos resolvió la inversión de sentido en pasillos de 2,5 m. Falta probar que el vehículo resuelva un giro de 90° en una esquina estrecha, que es lo que exige el guiado en edificios con más de un pasillo por piso.

Más de un vehículo por nivel y misiones simultáneas. El sistema atiende una misión a la vez, y cada nivel tiene un solo vehículo. Con esas dos condiciones la asignación es trivial (sección 2.3.1). En cuanto haya dos o más vehículos elegibles por tarea, o varias solicitudes a la vez, la asignación deja de ser trivial, y los métodos de subasta, como MURDOCH [44] o CBBA [45], empiezan a decidir algo. Es la extensión natural del trabajo para edificios con más afluencia.

Ampliar la muestra de las campañas. Con treinta misiones en simulación el intervalo de confianza mide 24 puntos porcentuales, y con nueve en los vehículos la cota inferior es del 70 %. Ese ancho es de diseño y solo se reduce con más repeticiones. Una campaña mayor permitiría además comparar las misiones de un mismo piso con las de dos pisos, comparación que hoy no se sostiene porque sus intervalos se solapan.

Evaluación con usuarios. Las cuatro métricas del proyecto miden el sistema, y no la experiencia de quien lo usa. Un estudio con personas, como el que hicieron Guerreiro et al. [11] con su robot guía, permitiría medirla. Interesa saber si la interfaz se entiende sin explicación, si la velocidad del vehículo resulta cómoda para caminar a su lado y cuánto tolera el usuario la espera en la escalera. Esa evaluación es también el paso previo a ofrecer el sistema a personas con discapacidad visual, que es la aplicación con más impacto que sugiere la literatura revisada.

REFERENCIAS BIBLIOGRÁFICAS

[1] M. Jiménez Muñoz, «Arquitectura cognitiva para integración de usuarios humanos dentro de un sistema de mapas multinivel en una plataforma robótica asistencial,» Tesis de mtría., Universidad Politécnica de Madrid, Dept. Ing. Industrial, Madrid, España, 2024.

[2] B. P. Gerkey y M. J. Matarić, «A Formal Analysis and Taxonomy of Task Allocation in Multi-Robot Systems,» *The International Journal of Robotics Research*, vol. 23, n.º 9, págs. 939-954, 2004. DOI: 10.1177/0278364904045564.

[3] G. T. Varghese et al., «Multi-Robot System for Mapping and Localization,» en *8th International Conference on Robotics and Automation Engineering (ICRAE)*, 2023, págs. 1-6. DOI: 10.1109/ICRAE59816.2023.10458504.

[4] T. Kim, G. Kang, D. Lee y D. H. Shim, «Development of an Indoor Delivery Mobile Robot for a Multi-Floor Environment,» *IEEE Access*, vol. 12, págs. 1-12, 2024. DOI: 10.1109/ACCESS.2024.3381489.

[5] Z. Song, S. Lou, R. Tan y C. Lv, «Robot-Elevator Interaction through IoT Devices for Autonomous Robot Multi-Floor Transition,» en *10th International Conference on Electrical Engineering, Control and Robotics (EECR)*, 2024, págs. 1-6. DOI: 10.1109/EECR60807.2024.10607245.

[6] S. Macenski, T. Foote, B. Gerkey, C. Lalancette y W. Woodall, «Robot Operating System 2: Design, Architecture, and Uses in the Wild,» *Science Robotics*, vol. 7, n.º 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074.

[7] K. Omer y A. Monteriu, «Multi-Layer Robotic Controller for Enhancing the Safety of Mobile Robot Navigation in Human-Centered Indoor Environments,» *Frontiers in Robotics and AI*, vol. 12, págs. 1-15, 2025. DOI: 10.3389/frobt.2025.00001.

[8] B. R. Nair, «Collaborative Perception in Multi-Robot Systems: Case Studies in Household Cleaning and Warehouse Operations,» en *6th International Conference on Robotics and Computer Vision (ICRCV)*, 2024, págs. 1-6. DOI: 10.1109/ICRCV60186.2024.00012.

[9] W. Burgard, A. B. Cremers, D. Fox, D. Hähnel, G. Lakemeyer, D. Schulz, W. Steiner y S. Thrun, «Experiences with an Interactive Museum Tour-Guide Robot,» *Artificial Intelligence*, vol. 114, n.º 1–2, págs. 3-55, 1999.

[10] D. Fox, W. Burgard, F. Dellaert y S. Thrun, «Monte Carlo Localization: Efficient Position Estimation for Mobile Robots,» en *Proceedings of the Sixteenth National Conference on Artificial Intelligence (AAAI)*, Orlando, FL, USA, 1999, págs. 343-349.

[11] J. Guerreiro, D. Sato, S. Asakawa, H. Dong, K. M. Kitani y C. Asakawa, «CaBot: Designing and Evaluating an Autonomous Navigation Robot for Blind People,» en *21st International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS)*, Pittsburgh, PA, USA, 2019, págs. 68-82. DOI: 10.1145/3308561.3353771.

[12] A. Li y H. Gurocak, «Modified Bug Algorithm with Proximity Sensors to Reduce Human-Cobot Collisions,» en *9th International Conference on Automation, Robotics and Applications (ICARA)*, 2023, págs. 1-6. DOI: 10.1109/ICARA56516.2023.10126056.

[13] A. Sojan, C. Thomas, B. Raj, S. A. Simon, S. K. John y A. A. Jose, «Autonomous Indoor Navigator,» en *11th International Conference on Smart Computing and Communication (ICSCC)*, 2025, págs. 1-6. DOI: 10.1109/ICSCC66177.2025.11233830.

[14] S. Macenski, F. Martín, R. White y J. Ginés Clavero, «The Marathon 2: A Navigation System,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Las Vegas, NV, USA, 2020. arXiv: 2003.00368.

[15] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov y M. Ferguson, «From the Desks of ROS Maintainers: A Survey of Modern & Capable Mobile Robotics Algorithms in the Robot Operating System 2,» *Robotics and Autonomous Systems*, vol. 168, pág. 104 493, 2023. DOI: 10.1016/j.robot.2023.104493.

[16] M. Bojarski et al., «End to End Learning for Self-Driving Cars,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Preprint, arXiv:1604.07316, Daejeon, South Korea, 2016.

[17] L. H. Zain, «Real-Time Obstacle Avoidance for a Mobile Robot Using CNN-Based Sensor Fusion,» en *International Conference on Robotics and Automation (ICRA)*, Preprint, arXiv:2509.08095, 2025.

[18] J. López Velásquez, G. A. Acosta Amaya y J. A. Jiménez Builes, «Enhancing Mobile Robot Navigation: Integrating Reactive Autonomy through Deep Learning and Fuzzy Behavior,» *Journal of Intelligent & Robotic Systems*, vol. 110, n.º 2, págs. 45-58, 2024. DOI: 10.1007/s10846-024-02045-2.

[19] E. Roszkowska, P. Makowski-Czerski y L. Janiec, «Multi-Level Control for Multiple Mobile Robot Systems,» en *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, págs. 1-7. DOI: 10.1109/ICRA48891.2023.10160734.

[20] B. Rudrusamy et al., «Multi-Robot Systems Using Broadcast Control Framework with Visual-Based Tracking,» en *7th International Conference on Mechanical Engineering and Robotics Research (ICMERR)*, 2022, págs. 1-6. DOI: 10.1109/ICMERR56497.2022.10097802.

[21] X. Xu, S. A. Prieto y B. García de Soto, «A Modular ROS Framework for Cooperative Multi-Robot Task Allocation in Digital Environments,» *MDPI Sensors*, vol. 25, n.º 4, págs. 1-20, 2025. DOI: 10.3390/s25040001.

[22] J. Zhan, X. Yu y T. Westerlund, «Mejorando la resiliencia de los sistemas multirrobóticos basados en ROS 2 con Kubernetes: un estudio de caso sobre el posicionamiento,» *MDPI Sensors*, vol. 25, n.º 16, pág. 5067, 2025. DOI: 10.3390/s25165067.

[23] G. A. Korsah, A. Stentz y M. B. Dias, «A Comprehensive Taxonomy for Multi-Robot Task Allocation,» *The International Journal of Robotics Research*, vol. 32, n.º 12, págs. 1495-1512, 2013. DOI: 10.1177/0278364913496484.

[24] A. K. Srivastava, J. M. Levin y P. Dames, «DELIVER: A System for Coordinated Multi-Robot Pickup and Delivery Using Relay Planning,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, págs. 1-8.

[25] A. E. Piguave Toala y D. S. Ronquillo Manosalvas, *Diseño e implementación de un sistema multi-robot de código abierto para ambientes colaborativos en ROS2*, Proyecto integrador, Fac. Ing. Electrónica, Escuela Superior Politécnica del Litoral, Guayaquil, Ecuador, 2023.

[26] S. Luo, Q. Yu y C. Zhu, «"Elevator or Stairs": A Layered Reinforcement Learning Approach for Robot Multi-Floor Navigation in High-Traffic Scenarios,» en *International Conference on Advanced Computing and Intelligent Robotics Applications (ACIRA)*, 2025, págs. 1-8. DOI: 10.1109/ACIRA67680.2025.11334740.

[27] J. Chen et al., «LITE: A Learning-Integrated Topological Explorer for Multi-Floor Indoor Environments,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, págs. 1-8. DOI: 10.1109/IROS60139.2025.11246317.

[28] Z. Gong et al., «Stairway to Success: An Online Floor-Aware Zero-Shot Object-Goal Navigation Framework via LLM-Driven Coarse-to-Fine Exploration,» *IEEE Robotics and Automation Letters*, vol. 11, n.º 3, págs. 1-8, 2026. DOI: 10.1109/LRA.2026.3655265.

[29] B. Kehoe, S. Patil, P. Abbeel y K. Goldberg, «A Survey of Research on Cloud Robotics and Automation,» *IEEE Transactions on Automation Science and Engineering*, vol. 12, n.º 2, págs. 398-409, 2015. DOI: 10.1109/TASE.2014.2376492.

[30] N. Tahir y R. Parasuraman, «Edge Computing and Its Application in Robotics: A Survey,» *MDPI Sensors*, vol. 25, n.º 2, págs. 1-25, 2025. DOI: 10.3390/s25020001.

[31] I. Malavolta et al., «Computation Offloading for Ground Robotic Systems Communicating over WiFi,» *Empirical Software Engineering*, vol. 28, n.º 3, págs. 1-35, 2023. DOI: 10.1007/s10664-022-10268-0.

[32] S. Thrun, W. Burgard y D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.

[33] D. V. Lu, D. Hershberger y W. D. Smart, «Layered Costmaps for Context-Sensitive Navigation,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Chicago, IL, USA, 2014, págs. 709-715.

[34] J. A. Reeds y L. A. Shepp, «Optimal Paths for a Car That Goes Both Forwards and Backwards,» *Pacific Journal of Mathematics*, vol. 145, n.º 2, págs. 367-393, 1990. DOI: 10.2140/pjm.1990.145.367.

[35] R. Siegwart, I. R. Nourbakhsh y D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2.ª ed. Cambridge, MA, USA: MIT Press, 2011.

[36] D. Dolgov, S. Thrun, M. Montemerlo y J. Diebel, «Path Planning for Autonomous Vehicles in Unknown Semi-structured Environments,» *The International Journal of Robotics Research*, vol. 29, n.º 5, págs. 485-501, 2010. DOI: 10.1177/0278364909359210.

[37] S. Macenski, S. Singh, F. Martín y J. Ginés, «Regulated Pure Pursuit for Robot Path Tracking,» *Autonomous Robots*, vol. 47, págs. 685-694, 2023. arXiv: 2305.20026.

[38] Object Management Group, «Data Distribution Service (DDS), Version 1.4,» Object Management Group, inf. téc. formal/15-04-10, 2015.

[39] C. Crick, G. Jay, S. Osentoski, B. Pitzer y O. C. Jenkins, «Rosbridge: ROS for Non-ROS Users,» en *Robotics Research*, ép. Springer Tracts in Advanced Robotics, H. I. Christensen y O. Khatib, eds., vol. 100, Cham, Switzerland: Springer, 2017. DOI: 10.1007/978-3-319-29363-9_28.

[40] L. D. Brown, T. T. Cai y A. DasGupta, «Interval Estimation for a Binomial Proportion,» *Statistical Science*, vol. 16, n.º 2, págs. 101-133, 2001. DOI: 10.1214/ss/1009213286.

[41] K. R. Popper, *The Logic of Scientific Discovery*. London, UK: Hutchinson, 1959.

[42] H. W. Kuhn, «The Hungarian Method for the Assignment Problem,» *Naval Research Logistics Quarterly*, vol. 2, n.º 1–2, págs. 83-97, 1955. DOI: 10.1002/nav.3800020109.

[43] R. G. Smith, «The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver,» *IEEE Transactions on Computers*, vol. C-29, n.º 12, págs. 1104-1113, 1980.

[44] B. P. Gerkey y M. J. Matarić, «Sold!: Auction Methods for Multirobot Coordination,» *IEEE Transactions on Robotics and Automation*, vol. 18, n.º 5, págs. 758-768, 2002.

[45] H.-L. Choi, L. Brunet y J. P. How, «Consensus-Based Decentralized Auctions for Robust Task Allocation,» *IEEE Transactions on Robotics*, vol. 25, n.º 4, págs. 912-926, 2009. DOI: 10.1109/TRO.2009.2022423.

[46] S. Macenski, M. Booker y J. Wallace, *Open-Source, Cost-Aware Kinematically Feasible Planning for Mobile and Surface Robotics*, 2024. arXiv: 2401.13078.

[47] R. C. Coulter, «Implementation of the Pure Pursuit Path Tracking Algorithm,» Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, USA, inf. téc. CMU-RI-TR-92-01, 1992.

[48] M. Jaimez, J. G. Monroy y J. González-Jiménez, «Planar Odometry from a Radial Laser Scanner. A Range Flow-Based Approach,» en *IEEE International Conference on Robotics and Automation (ICRA)*, Stockholm, Sweden, 2016, págs. 4479-4485. DOI: 10.1109/ICRA.2016.7487647.

[49] J. Zhang, M. Kaess y S. Singh, «On Degeneracy of Optimization-Based State Estimation Problems,» en *IEEE International Conference on Robotics and Automation (ICRA)*, Stockholm, Sweden, 2016, págs. 809-816.

[50] S. Macenski e I. Jambrecic, «SLAM Toolbox: SLAM for the Dynamic World,» *Journal of Open Source Software*, vol. 6, n.º 61, pág. 2783, 2021. DOI: 10.21105/joss.02783.

[51] T. Moore y D. Stouch, «A Generalized Extended Kalman Filter Implementation for the Robot Operating System,» en *13th International Conference on Intelligent Autonomous Systems (IAS-13)*, Padova, Italy, 2014.

[52] N. Koenig y A. Howard, «Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator,» en *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Sendai, Japan, 2004, págs. 2149-2154.

[53] E. B. Wilson, «Probable Inference, the Law of Succession, and Statistical Inference,» *Journal of the American Statistical Association*, vol. 22, n.º 158, págs. 209-212, 1927. DOI: 10.1080/01621459.1927.10502953.

[54] ASTM International, «Standard Test Method for Navigation: Defined Area,» ASTM International, West Conshohocken, PA, USA, inf. téc. ASTM F3244-21, 2021.

[55] Departamento Nacional de Planeación, «Documento CONPES 3975: Política Nacional para la Transformación Digital e Inteligencia Artificial,» Consejo Nacional de Política Económica y Social, Bogotá D.C., Colombia, inf. téc., 2019.

[56] Departamento Nacional de Planeación, «Documento CONPES 4069: Política Nacional de Ciencia, Tecnología e Innovación 2022–2031,» Consejo Nacional de Política Económica y Social, Bogotá D.C., Colombia, inf. téc., 2021.

[57] ICONTEC, «NTC 1486: Documentación. Presentación de tesis, trabajos de grado y otros trabajos de investigación,» Instituto Colombiano de Normas Técnicas y Certificación, Bogotá D.C., Colombia, inf. téc., 2022.

[58] Donkey Car Project, *Donkey Car: an open source DIY self driving platform for small scale cars*, `https://www.donkeycar.com`, Consultado en abril de 2026.

[59] M. O'Kelly, V. Sukhil, H. Abbas et al., *F1/10: An Open-Source Autonomous Cyber-Physical Platform*, arXiv:1901.08567, 2019.

[60] AWS DeepRacer, *aws-deepracer: simulation artifacts, core components and configuration files to integrate the AWS DeepRacer device with the ROS Navigation stack*, Repositorio de GitHub, `https://github.com/aws-deepracer/aws-deepracer`, 2021.

[61] AWS DeepRacer Community, *deepracer-custom-car: custom software stack for the AWS DeepRacer*, Repositorio de GitHub, `https://github.com/aws-deepracer-community/deepracer-custom-car`, Consultado en mayo y agosto de 2026.

[62] Nav2 Project, *First-Time Robot Setup Guide*, Documentación de Nav2, `https://docs.nav2.org`, Consultado en septiembre de 2026.